CMP 519 Software Engineering

UNIT 04 – Object Oriented Analysis and Design with UML

Instructor: Dr. Suresh Pokharel

DISCLAIMER

These slides are part of teaching materials for Software Engineering. These slides do not cover all aspect of learning Software Engineering, nor are these be taken as primary source of information. As the core textbooks and reference books for learning the subject has already been specified and provided to the students, students are encouraged to learn from the original sources.

OBJECT ORIENTED PROGRAMMING (OOP)

"Object-oriented programming is a method of implementation in which programs are organized as cooperative **collections of objects**, each of which represents **an instance of some class**, and whose classes are all members of a **hierarchy of classes** united via **inheritance relationships**. "

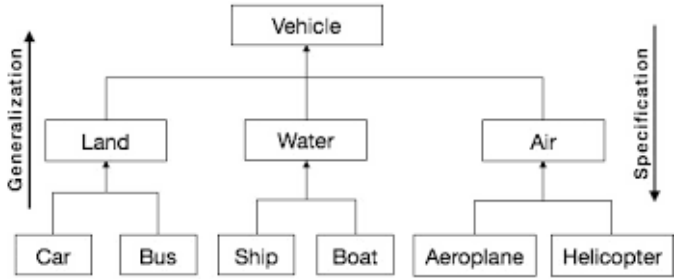
- uses objects, not algorithms, as its fundamental logical building blocks
- each object is an instance of some class
- classes may be related to one another via inheritance relationships

Booch et al., *Object-Oriented Analysis and Design with Applications*

OBJECT ORIENTED PROGRAMMING Vs PROCEDURAL PROGRAMMING

PROCEDURAL	OBJECT ORIENTED PROGRAMMING
Program is divided into small parts: functions	Program is divided into small parts: objects
Follows top-down approach	Follows bottom-up approach
No access specifier	Access specifier: private, public, protected
No proper way for data hiding: less secure	Data hiding: more secure
Function is more important than data	Data is more important than function
Not simulation of real world	Simulation of real world
FORTRAN, ALGOL, COBOL, BASIC, Pascal, C.	Java, C++, C#, Python, PHP, JavaScript, Ruby, Perl, Objective-C, Dart, Swift, Scala.

OBJECT ORIENTED MODELS



References:

1. Booch et al., *Object-Oriented Analysis and Design with Applications*
2. *Internet*

Simplification of real world

BENEFITS

- **Simplification**: It simplifies complex system. It provides the template that guide us better structuring complex system.
- **Understandably**: It helps us to specify the structure or behavior of the system
- **Fast development**: It helps in faster development of software.
- **Maintainability**: It is easy to maintain. Suppose a module develops an error, then a programmer can fix that particular module, while the other parts of the software are still up and running.
- **Visualization**: It visualizes a system as it is or as we want it to be
- **Reusability**: It enables reuse of objects, designs, and functions.
- **Better risk management:** It reduces development risks, particularly in integration of complex systems.

DIFFERENT LEVELS OF MODELS

- **Conceptual:** captures the system in terms of the domain entities that exist (or will exist) and their association with other such entities of your system.
- **Logical:** takes the concepts created in the conceptual model and establishes the existence and meaning of the key abstractions and mechanisms that will determine the system's architecture and overall design.
- **Physical:** describes the concrete software and hardware composition of the system's implementation. Obviously, the physical model is technology-specific.

We study different modeling from the object-oriented perspective

OBJECT ORIENTED MODEL

- Classes
- Objects

OO: CLASSES AND OBJECTS

How to convert real word world entities into software objects?

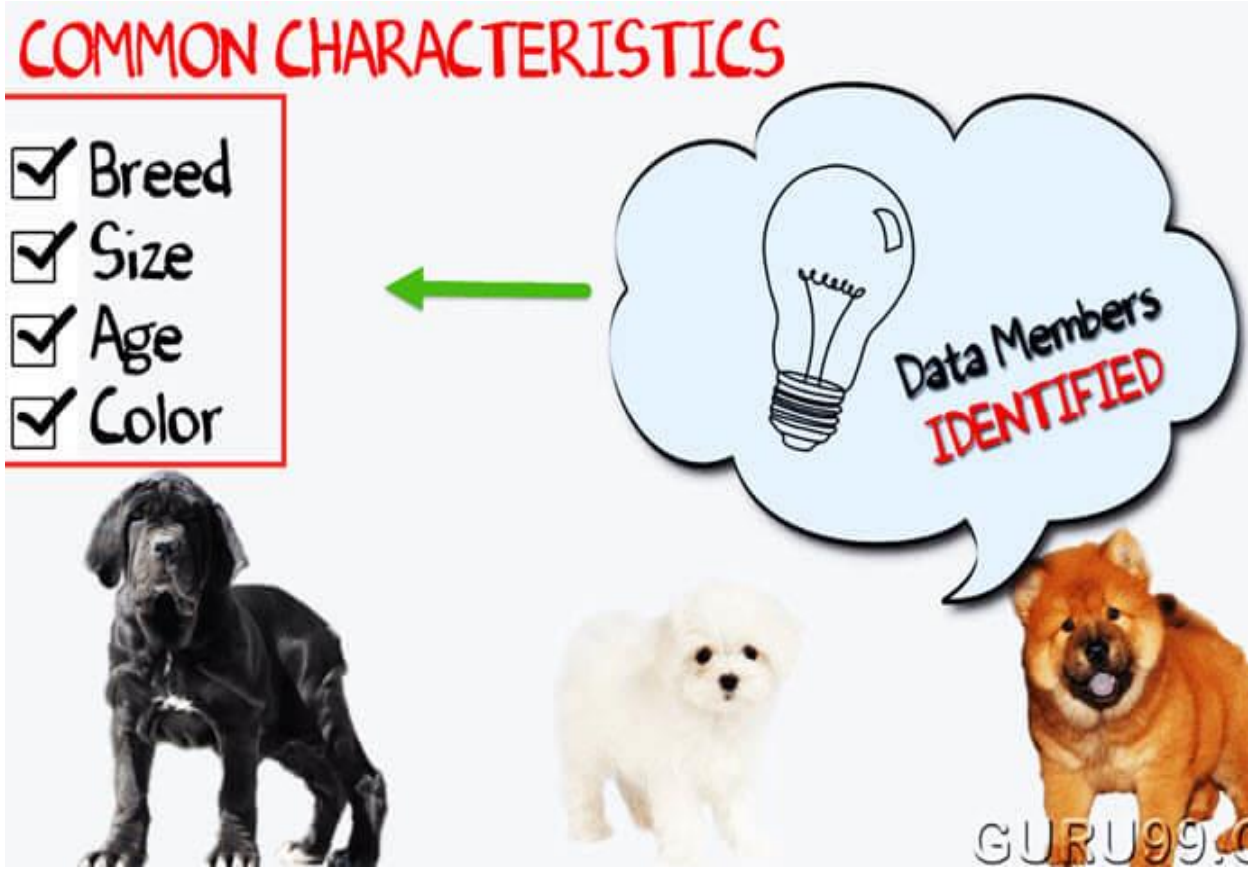
?

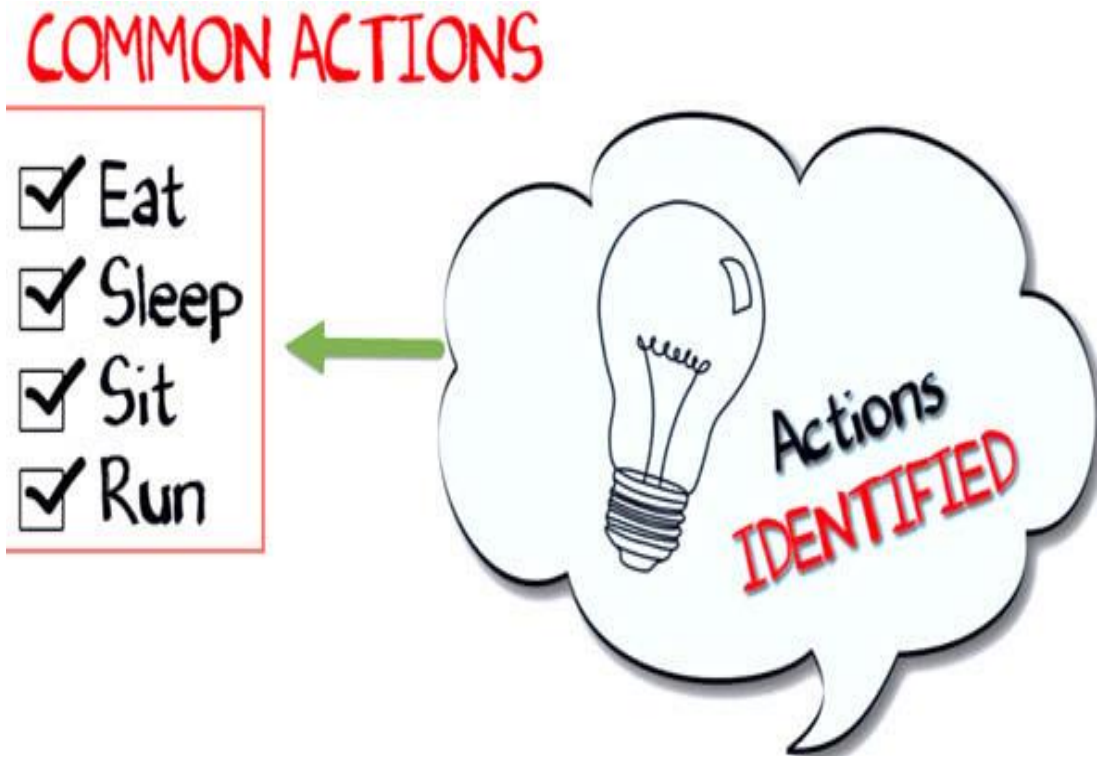
software objects

OO: CLASSES AND OBJECTS

Reference: https://www.guru99.com/java-oops-class-objects.html

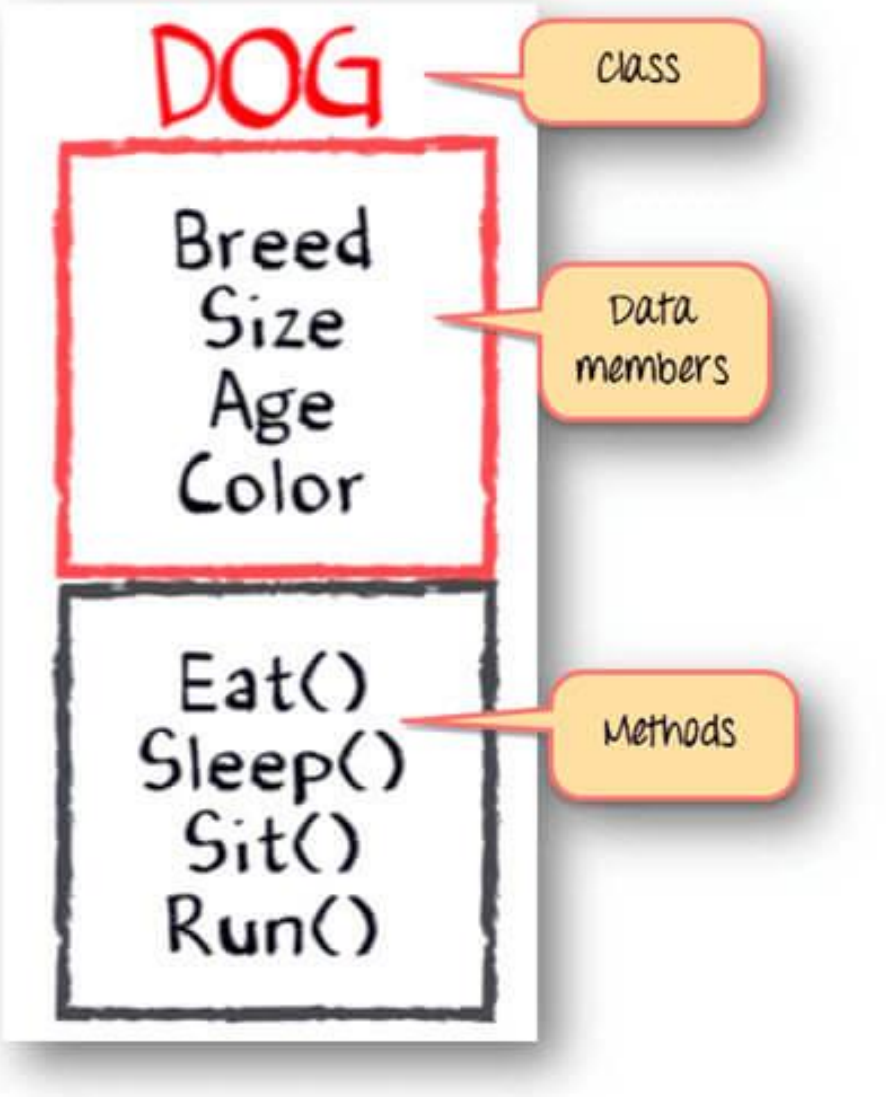
OO: CLASSES AND OBJECTS





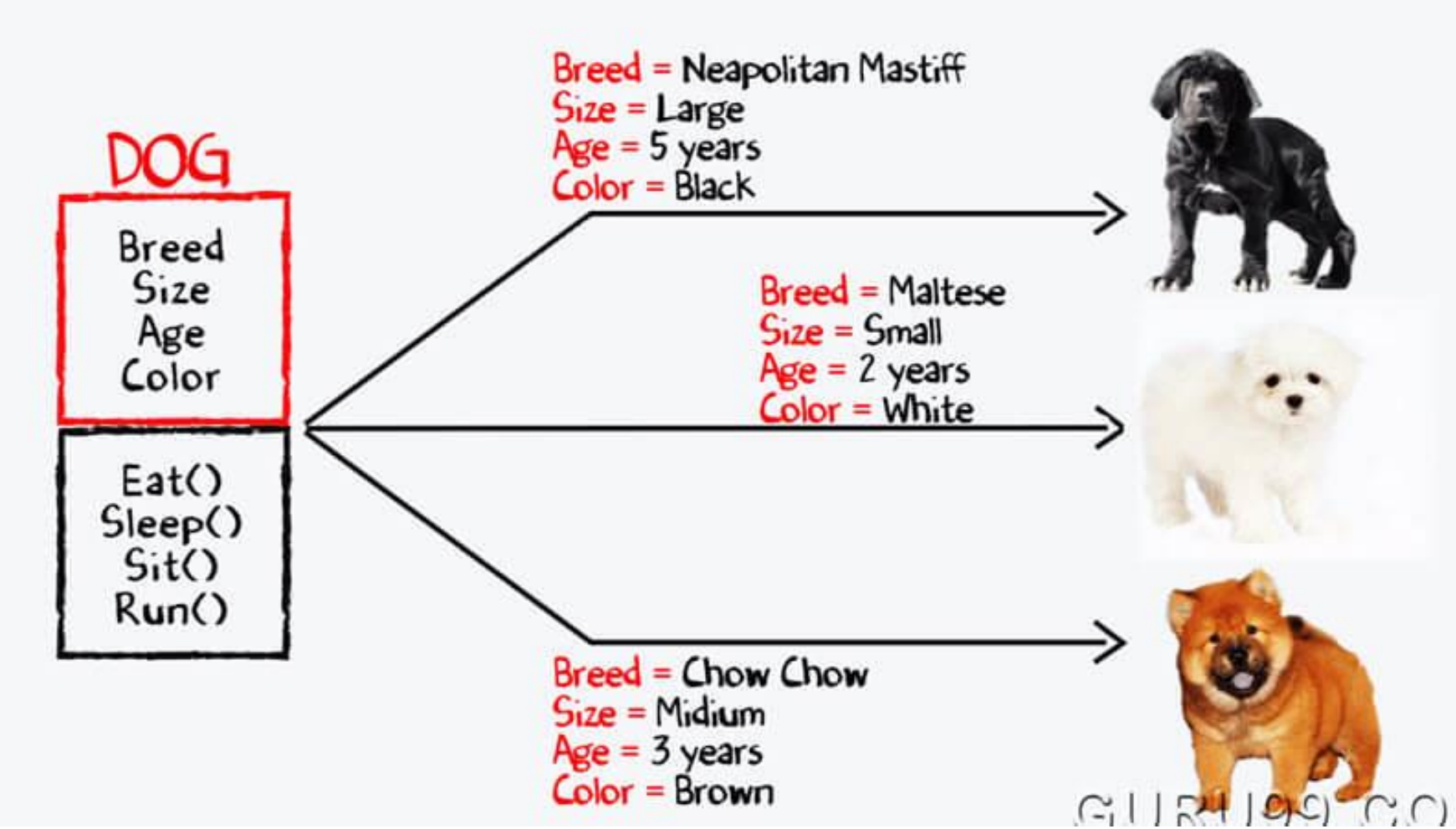
Reference: https://www.guru99.com/java-oops-class-objects.html

OO: CLASSES AND OBJECTS



Reference: https://www.guru99.com/java-oops-class-objects.html

OO: CLASSES AND OBJECTS



Reference: https://www.guru99.com/java-oops-class-objects.html

OO: CLASSES AND OBJECTS

```
// Class Declaration
public class Dog {
    // Instance Variables
    String breed;
    String size;
    int age;
    String color;


    // method 1
    public String getInfo() {
        return ("Breed is: "+breed+" Size is:"+size+" Age is:"+age+" color
is: "+color);
    }


    public static void main(String[] args) {
        Dog maltese = new Dog();
        maltese.breed="Maltese";
        maltese.size="Small";
        maltese.age=2;
        maltese.color="white";
        System.out.println(maltese.getInfo());
    }
}
```

Java Code

Reference: https://www.guru99.com/java-oops-class-objects.html

OO: CLASSES AND OBJECTS

Class

- Basic concept of OOP which represent real world entities. E.g Dog
- A blueprint or a set of instructions to build a specific type of object

Object

- An instance of a class. E.g Neapolitan Mastiff
- An object has state, behavior, and identity
- The structure and behavior of similar objects are defined in their common class
- The terms instance and object are interchangeable

ELEMENTS OF OBJECT MODEL

Major elements

- Abstraction
- Encapsulation
- Hierarchy
- Modularity

By major, we mean that a model without any one of these elements is not object oriented.

Minor elements

- Typing
- Concurrency
- Persistence

By minor, we mean that each of these elements is a useful, but not essential, part of the object model.

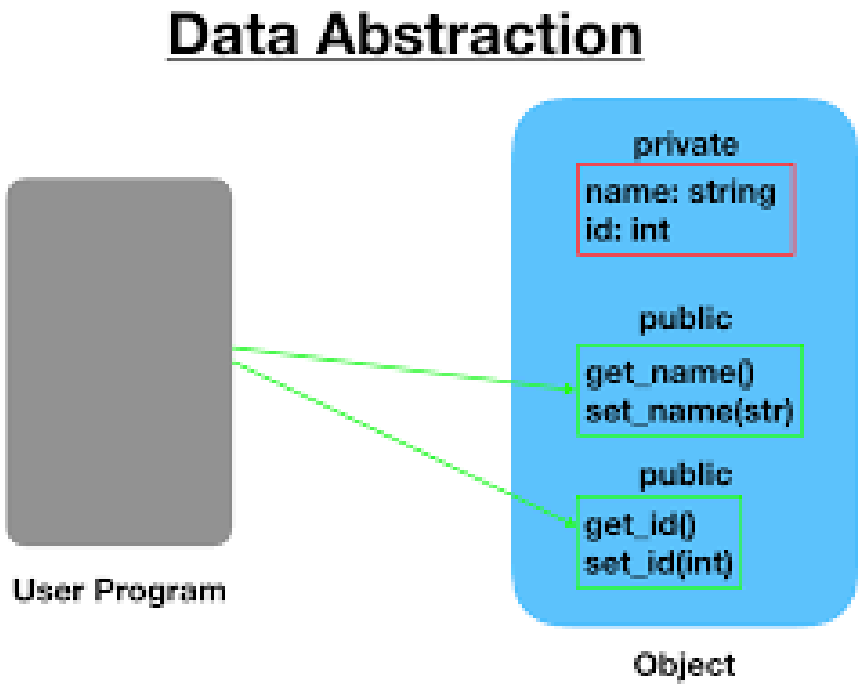
OO: ABSTRACTION

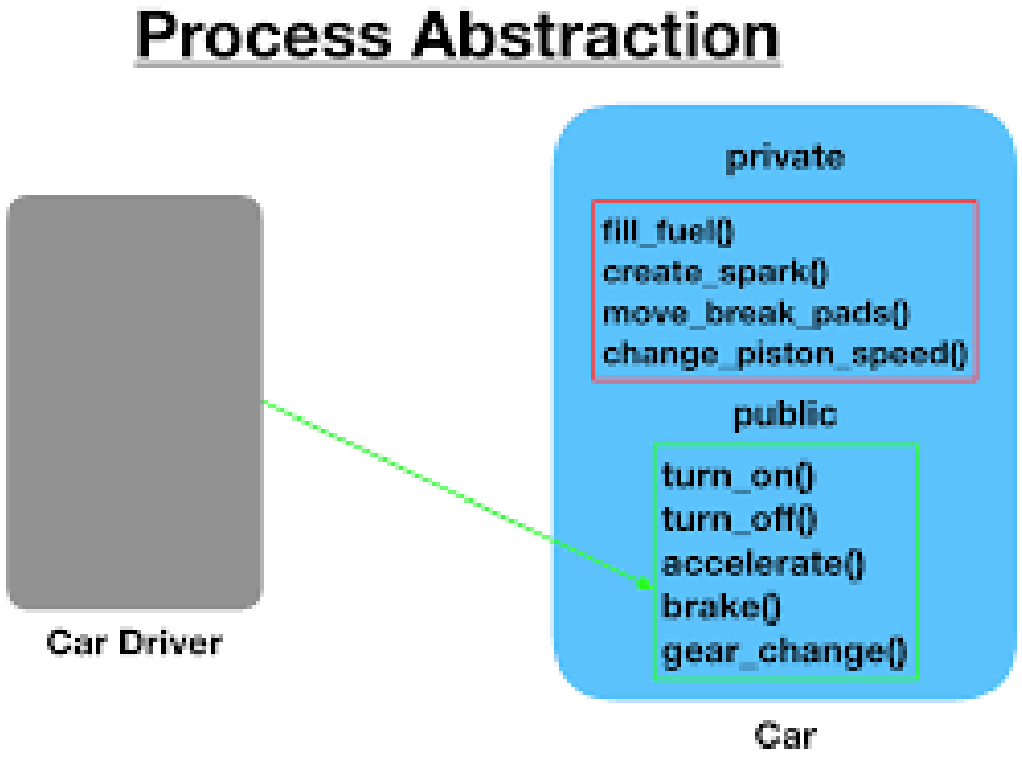
- Process of hiding the internal data and implementation details form the outer world
- Can be used to create boundary between the application and the client programs

Real world example: Your car is a great example of abstraction. You can start a car by turning the key or pressing the start button. You don't need to know how the engine is getting started, what all components your car has. The car internal implementation and complex logic is completely hidden from the user.

- Two types
  - Data abstraction
  - Process abstraction

OO: ABSTRACTION





Access Modifier: Private, Public, Protected

https://www.journaldev.com/33191/what-is-abstraction-in-oops

OO: ENCAPSULATION

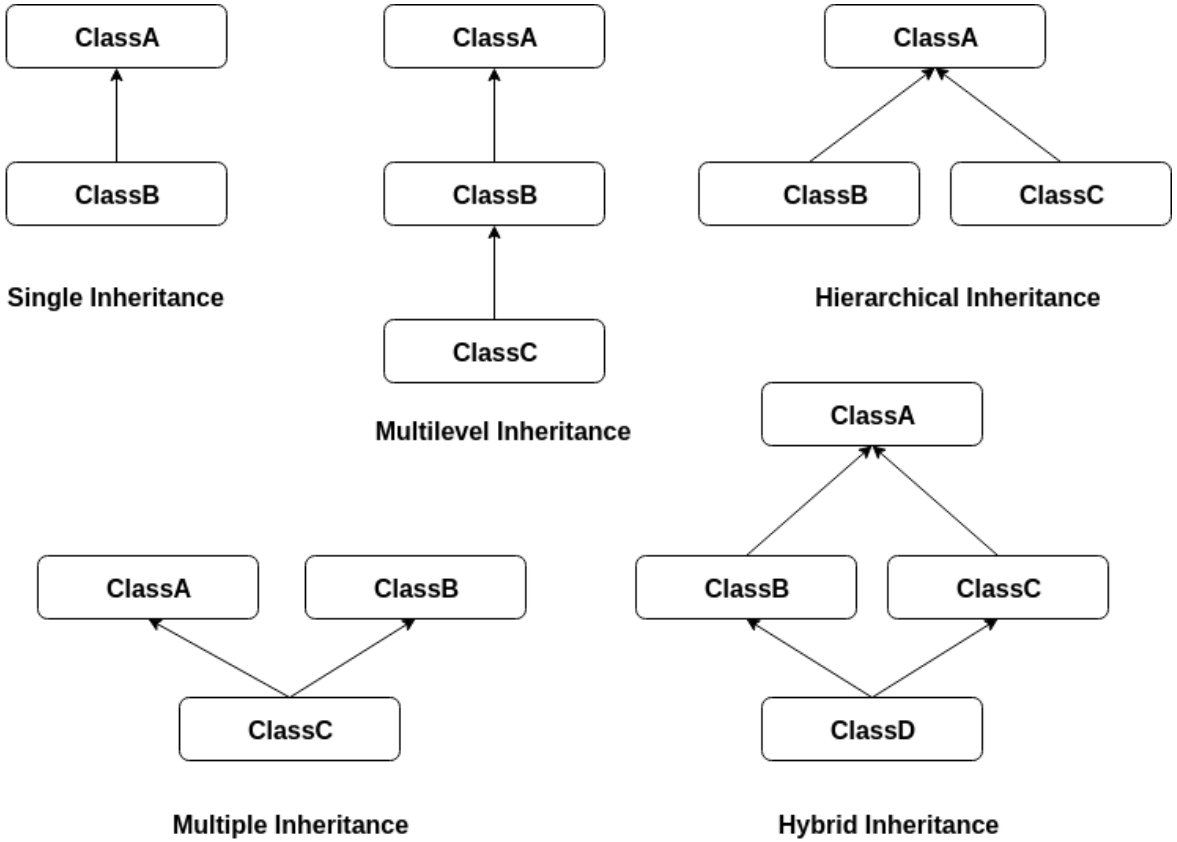
- Wrapping up variables, properties and methods altogether into single unit (class).
- Keeps data safe from outside interference and misuse
- Data hiding
- A ***class*** is a program-code-template that allows developers to create an object that has both variables (data) and behaviors (functions or methods).
- A class is an example of encapsulation in computer science in that it consists of data and methods that have been bundled into a single unit.
- Access modifiers are useful for data hiding

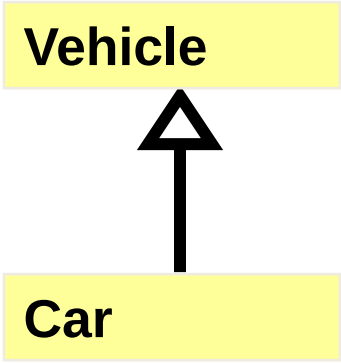
OO: HIERARCHY

- Used to organize and create reusable classes through inheritance
- *Inheritance* allows a software developer to derive a new class from an existing one
- The existing class is called the *parent class,* or *superclass*, or *base class*
- The derived class is called the *child class* or *subclass*.
- As the name implies, the child inherits characteristics of the parent
- That is, the child class inherits the methods and data defined for the parent class

OO: INHERITENCE

- Inheritance should create an *is-a relationship*, meaning the child *is a* more specific version of the parent

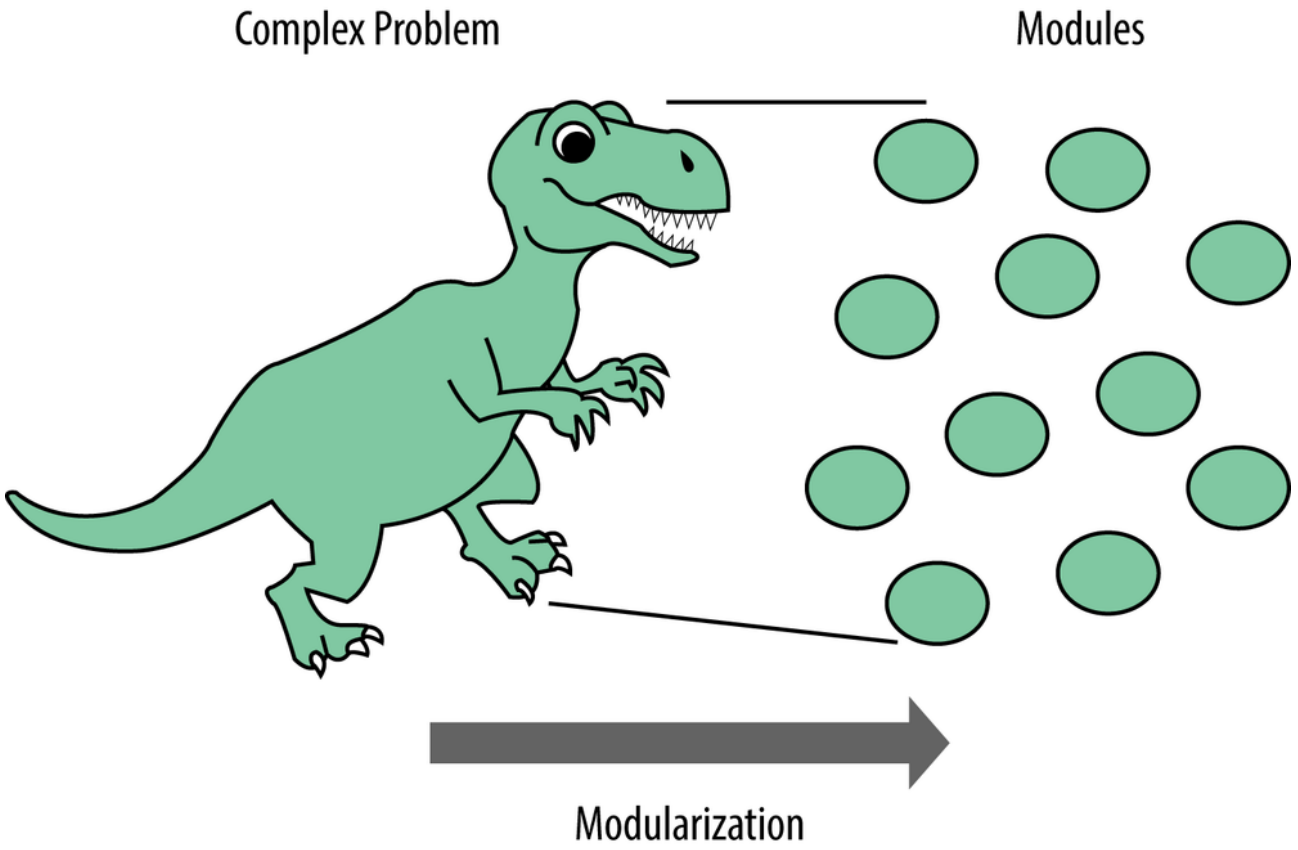





```
class Car extends Vehicle
{
    // class contents
}
```

OO: MODULARITY

- Decomposing a problem into small subproblems
- Modularity is the property of a system that has been decomposed into a set of cohesive and loosely coupled modules.



OO: TYPING, CONCURRENCY, PERSISTENCE

- **Typing:** precise characterization of structural or behavioral properties (enforcement of the class of an object), static typing, dynamic typing (polymorphism-see next slide)

- **Concurrency:** Concurrency is the property that distinguishes an active object from one that is not active.

- **Persistence:** state and class of an object is consistent across time or space

Analysis Object Models and Dynamic Models

Object Models and Dynamic Models

- **Object Models**: Represent the structure and relationships of objects within a system.
- **Dynamic Models**: Depict the behavior of a system as it responds to events.
- Key Components:
  - *Classes, attributes, and associations* (Object Model)
  - *States, transitions, and events* (Dynamic Model)
- Benefits:
  - Improved System Understanding: Visualize structure and behavior.
  - Better System Design: Provides a blueprint for implementation.
  - Enhanced Reusability and Maintainability: Encourages modular design.

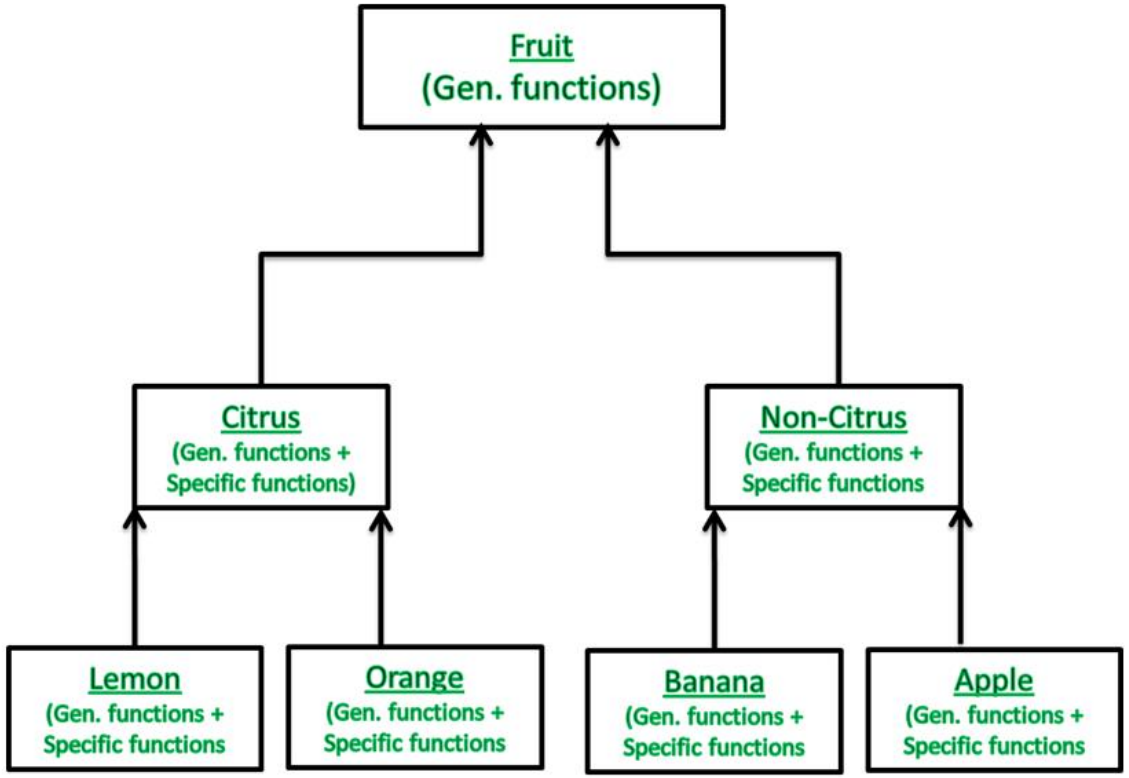
Entity, Boundary, and Control Objects

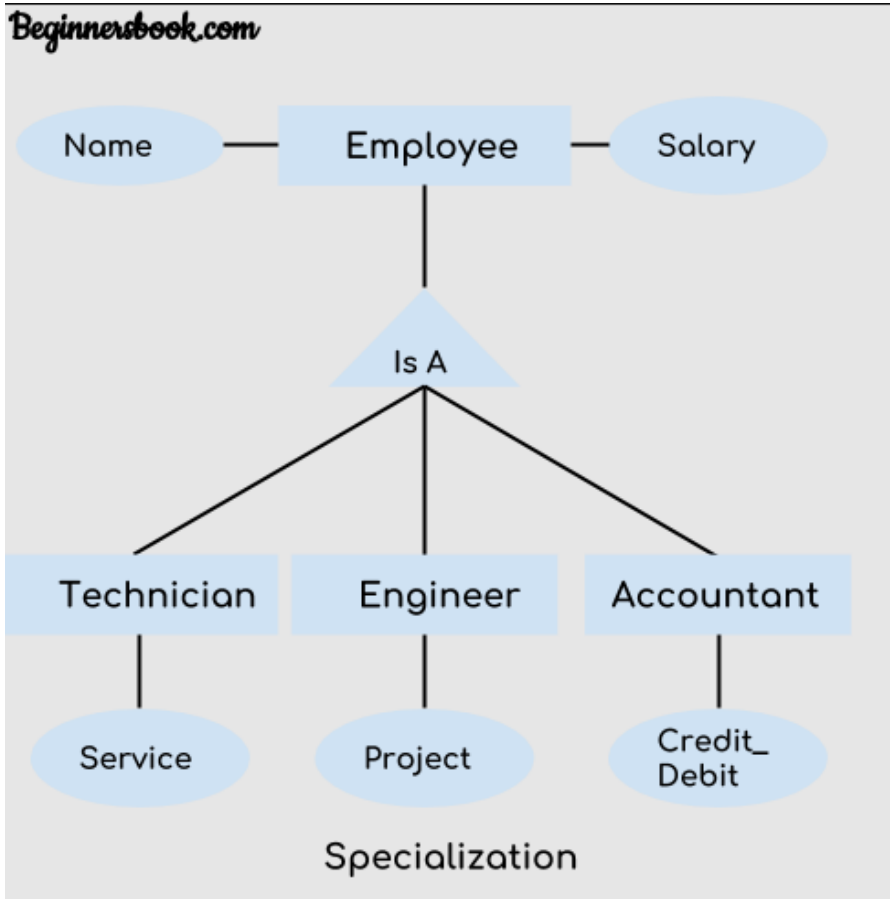
- *Entity Objects:*
  - Represent persistent information in the system.
  - Example: Customer, Order, Product.
- *Boundary Objects:*
  - Interface between the system and its environment.
  - Example: User interface forms or web pages.
- *Control Objects:*
  - Manage the flow of interactions between entity and boundary objects.
  - Example: Order processing controller.

Generalization and Specialization

- **Generalization**: Defines a hierarchy where a superclass represents common attributes and behavior.
  - Example: Vehicle (superclass) with Car and Motorcycle (subclasses).
- **Specialization**: Extends the functionality of a superclass to specific subclasses.
  - Example: Car extends Vehicle with additional properties.

Generalization and Specialization





UNIFIED MODELING LANGUAGE (UML)

Partial References:

1. Booch et al., *Object-Oriented Analysis and Design with Applications*
2. Lecture notes-OOSE, Pramod Parajuli, 2018
3. https://www.tutorialspoint.com/uml/index.htm
4. https://www.cs.uah.edu/~rcoleman/Common/SoftwareEng/UML.html

UML

- Unified modeling language (UML) for visualizing, specifying, constructing, documenting of artifact of a software system
- The blueprint of a system
- It is standard for building object oriented and component-based software system
- UML is a notation system though which we can visualize a model of a system
- It describe only design or structure of system

UML

- In early stages of model driven development there was a great need for some universal approach for modeling a software system
- As deferent people understand the notation in different way if there is no universal model approach
- At that time brooch, rumba, Jacobson gave their theories and universally accepted
- All of three decided to build a notation language by merging all of three language and out product of that was UML (unified modeling language)
- In 1997 OMG (Object management group) a non profit organization standardized the UML
- Since then, they are making improvement in UML
- Released versions of UML

SOFTWARES FOR UML

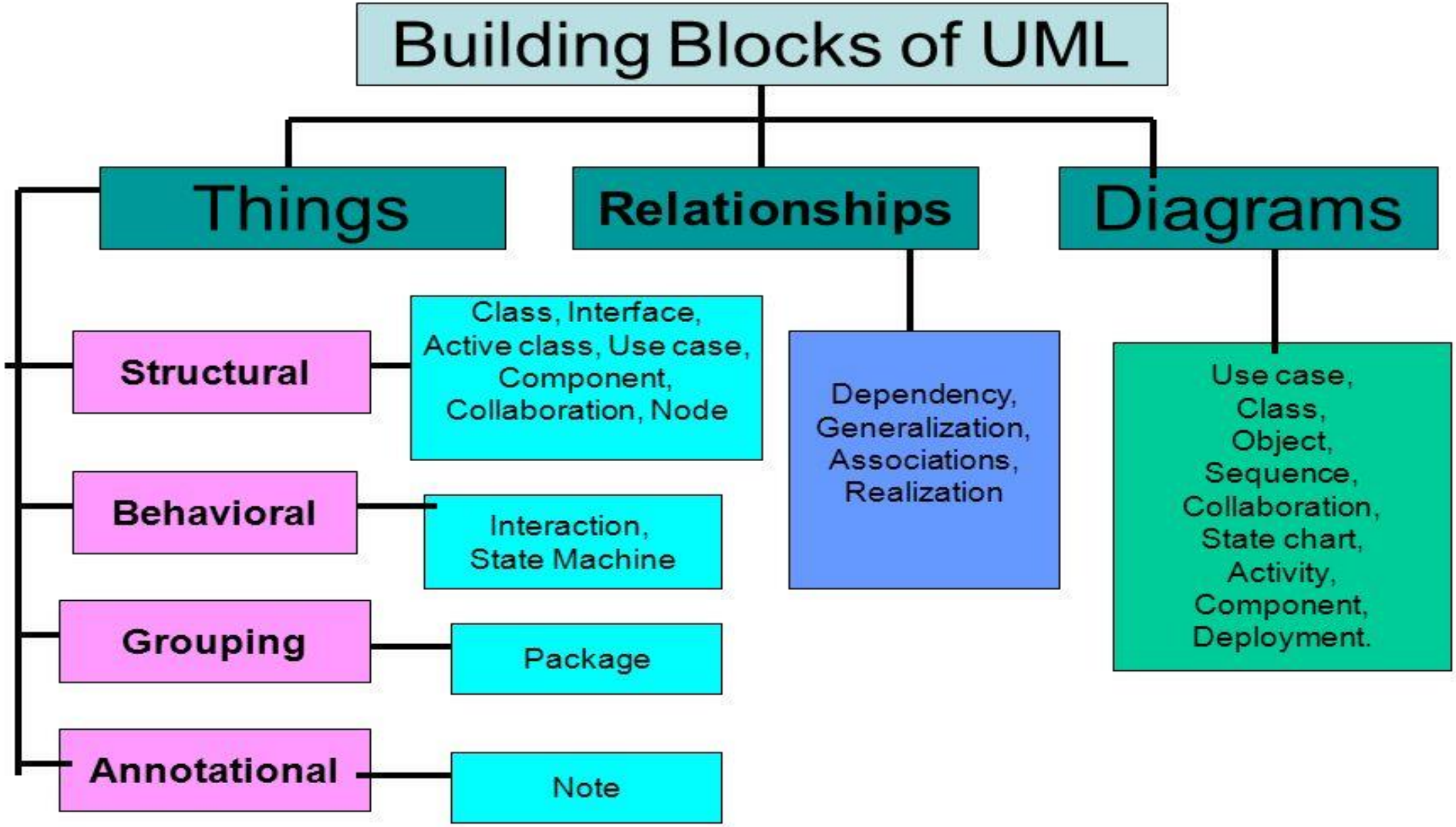
- https://en.wikipedia.org/wiki/List_of_Unified_Modeling_Language_tools
- https://www.predictiveanalyticstoday.com/open-source-free-unified-modeling-language-uml-tools/
- https://www.guru99.com/best-uml-tools.html

BUILDING BLOCKS OF UML

a) **Things-** These are the abstractions that are first-class citizens in a model.

b) **Relationships-** Tie the above things together.

c) **Diagrams-** Group interesting collections of things



https://slideplayer.com/slide/9873057/

THINGS

There are 4 kinds of things in the UML

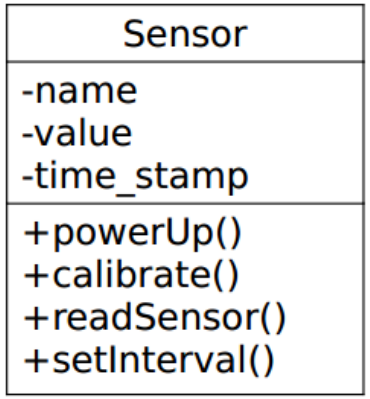
1. Structural Things
2. Behavioral Things
3. Grouping Things
4. Annotational Things

STRUCTURAL THINGS

- Structural things define the static part of the model. (*Noun* part of models)
- They represent the physical and conceptual elements.
  - Class
  - Interface
  - Collaboration
  - Use case
  - Active class
  - Component
  - Node

STRUCTURAL THINGS

Class – Class represents a set of objects having similar responsibilities.

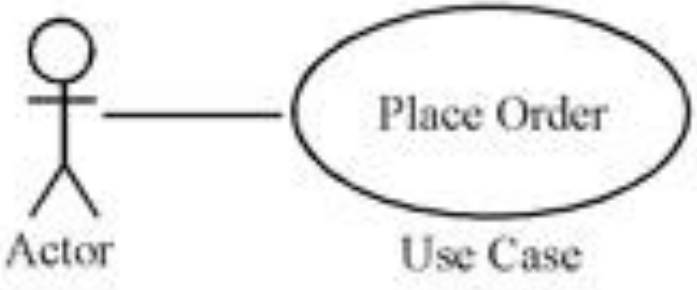


Interface – externally visible behavior of an element that specify a service of a class or component. Interface defines only the specifications not the implementations!

STRUCTURAL THINGS

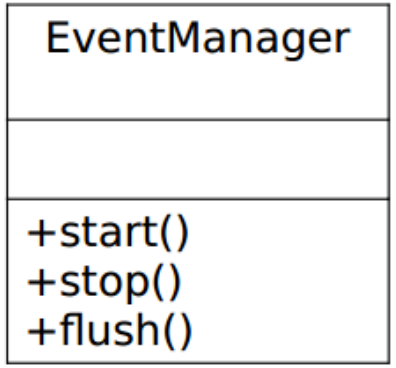
Collaboration - specify structure and behavior of collaboration between elements. We visualize such collaborations in use-case diagrams and activity diagrams.

Use case – Use case represents a set of actions performed by a system for a specific goal.

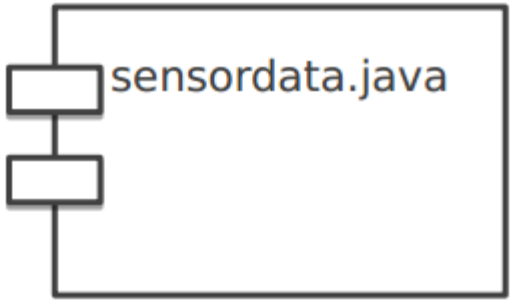


STRUCTURAL THINGS

Active Class - a class that its objects represent elements whose behavior is concurrent with other elements. i.e., threading.

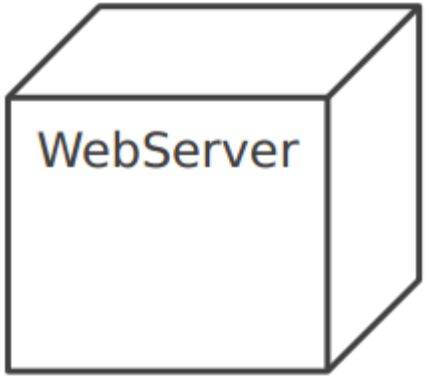


Component – physical and replaceable part that represents the packaging of logical elements such as classes, interfaces and collaborations.



STRUCTURAL THINGS

Node - exists at runtime and represents a computational resource. It contains a set of components.



BEHAVIORAL THINGS

- Represent dynamic components in UML models, the verb part of UML models

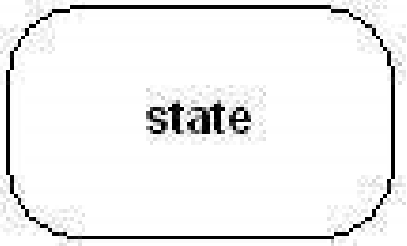
Two types;

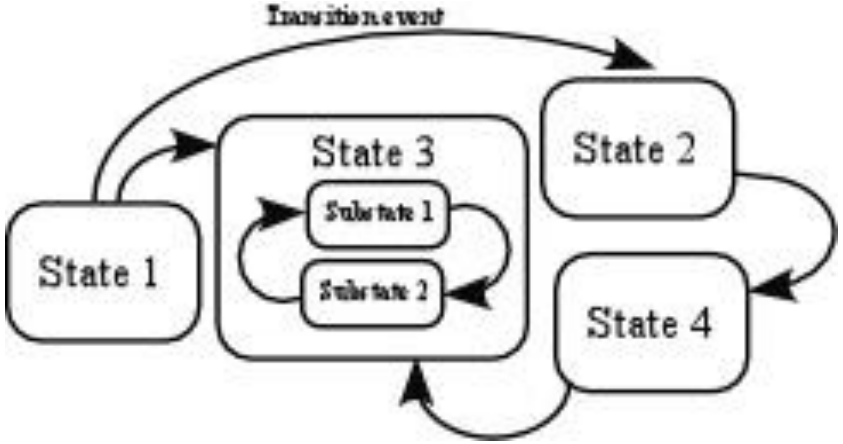
- Interaction
- State machine

BEHAVIORAL THINGS

Interaction - consists of a group of messages exchanged among elements to accomplish a specific task.

State Machine - State machine is useful when the state of an object in its life cycle is important. It defines the sequence of states an object goes through in response to events. Events are external factors responsible for state change



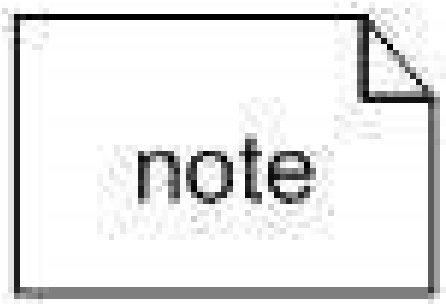


GROUPING THINGS

Package – Package is the only one grouping thing available for gathering structural and behavioral things. organize elements into groups

ANNOTATIONAL THINGS

Note - A note is used to render comments, constraints, etc. of an UML element.



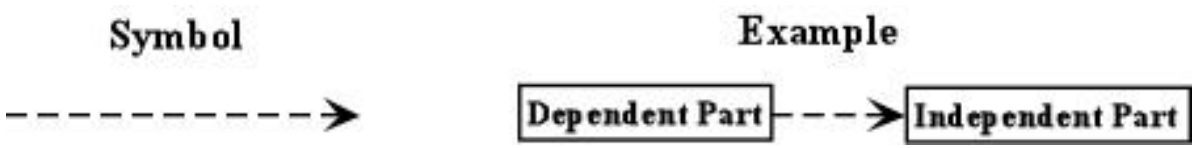
RELATIONAL

It shows how the elements are associated with each other and this association describes the functionality of an application. They are:

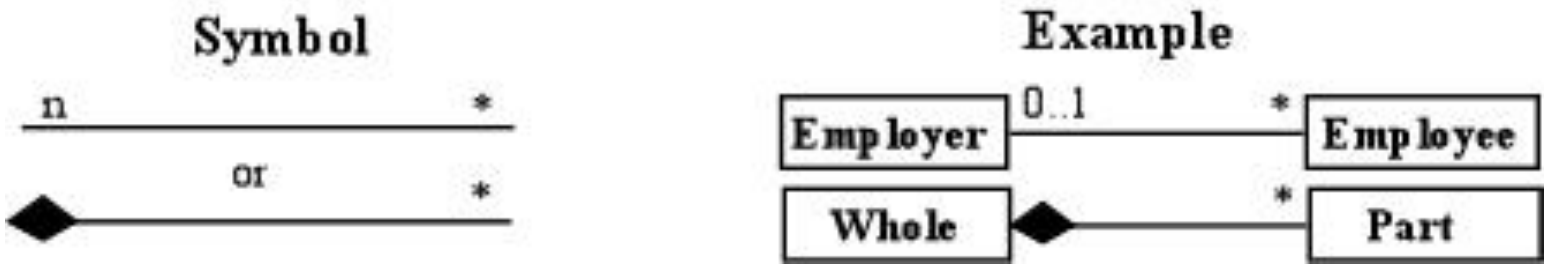
- Dependency
- Association
- Generalization
- Realization

RELATIONAL

Dependency - relationship between two things in which change in one element also affects the other.

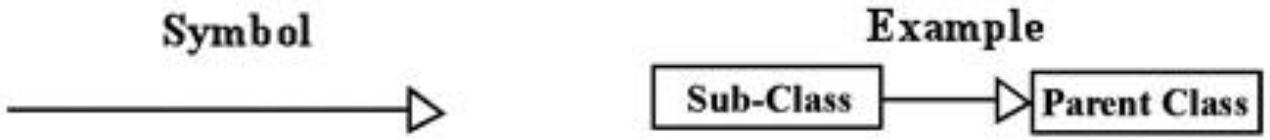


Association - a set of links that connects the elements of a UML model. It also describes how many objects are taking part in that relationship.

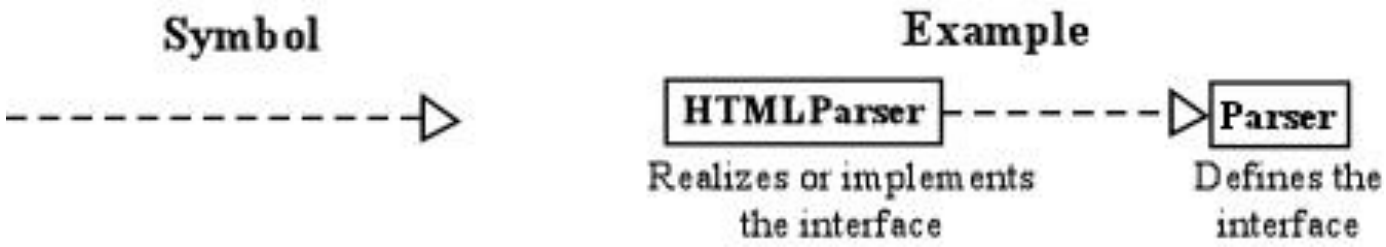


RELATIONAL

Generalization - a relationship which connects a specialized element with a generalized element. It basically describes the inheritance relationship in the world of objects.

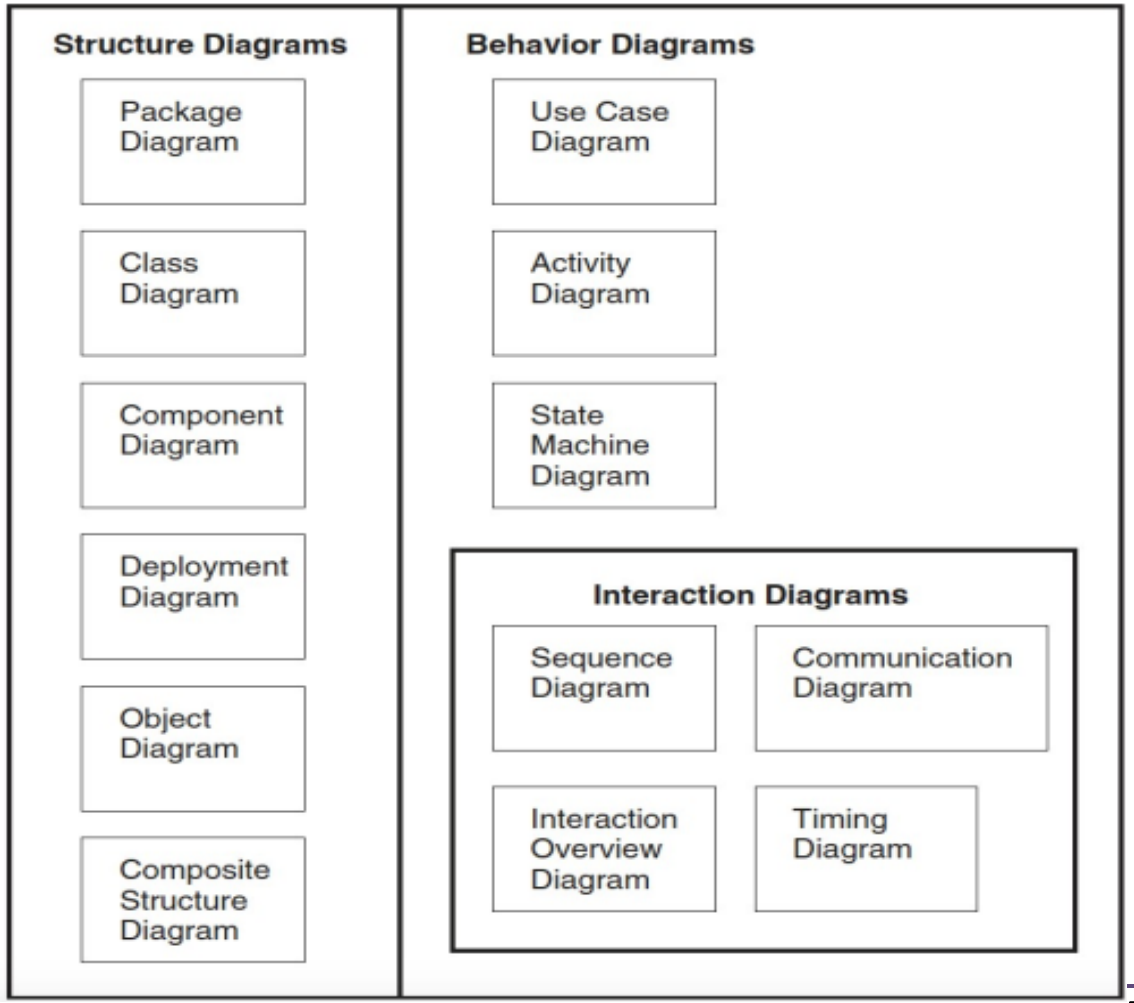


Realization - a relationship in which two elements are connected. One element describes some responsibility, which is not implemented and the other one implements them. This relationship exists in case of interfaces.



DIAGRAMS

- Diagrams are the ultimate output of the entire discussion
- All the elements, relationships are used to make a complete UML diagram and the diagram represents a system.

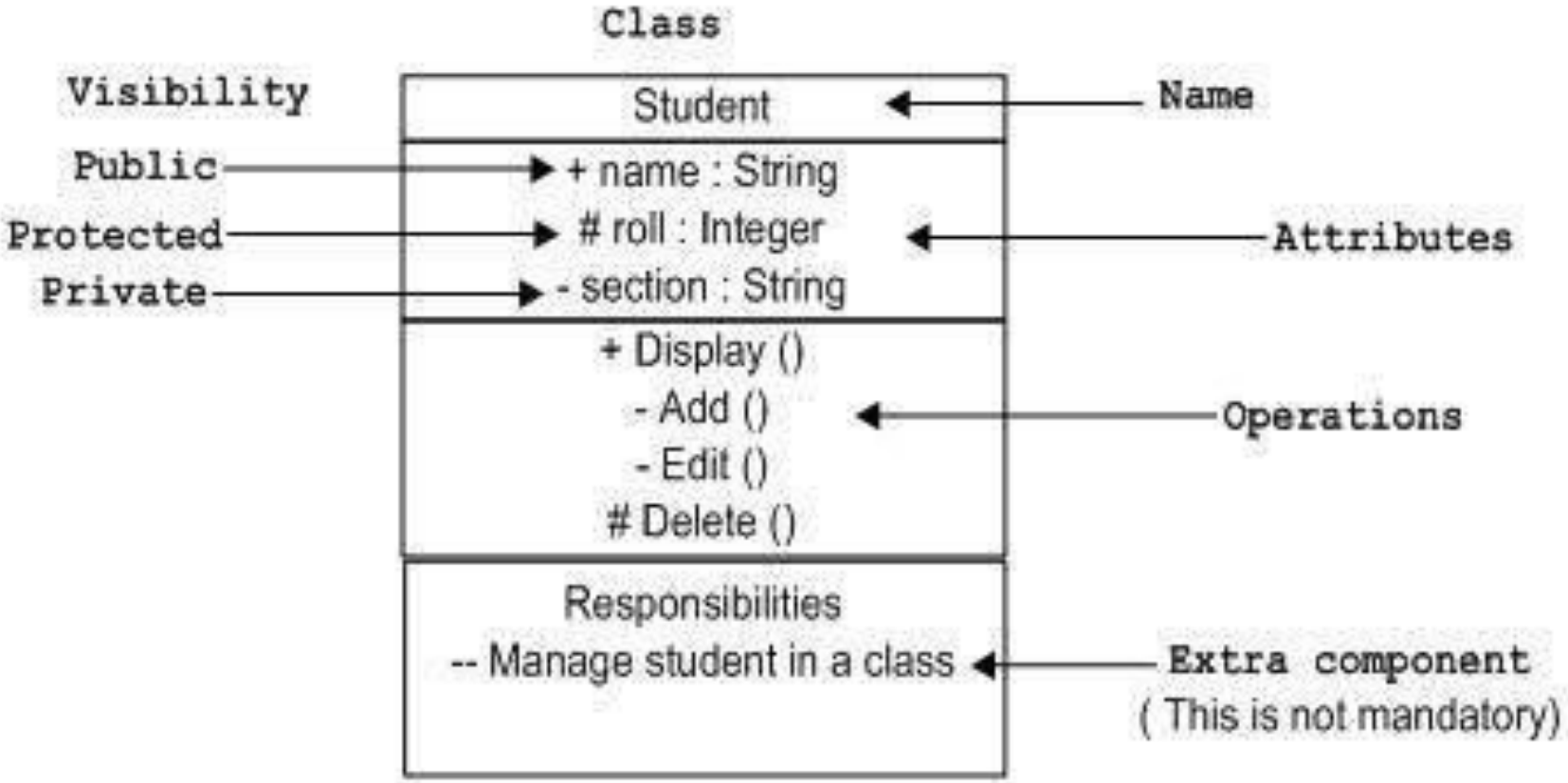


UML diagrams	Process segments				
	Requirements	Analysis	Design	Development	Deployment
Use case diagram	Sponsor objectives	Conceptual model			
Class diagram		Conceptual level classes	Specification level classes	Implementation level classes	
Object diagram		Identify objects	Identify objects		
State diagram		Object behavior			
Sequence diagram		Object interaction			
Collaboration diagram		Object interaction			
Activity diagram	User operation(s)		System operations		
Component diagram			Visualise components		
Package diagram	System requirements				
Deployment diagrams	System elements	System integration	System integration		

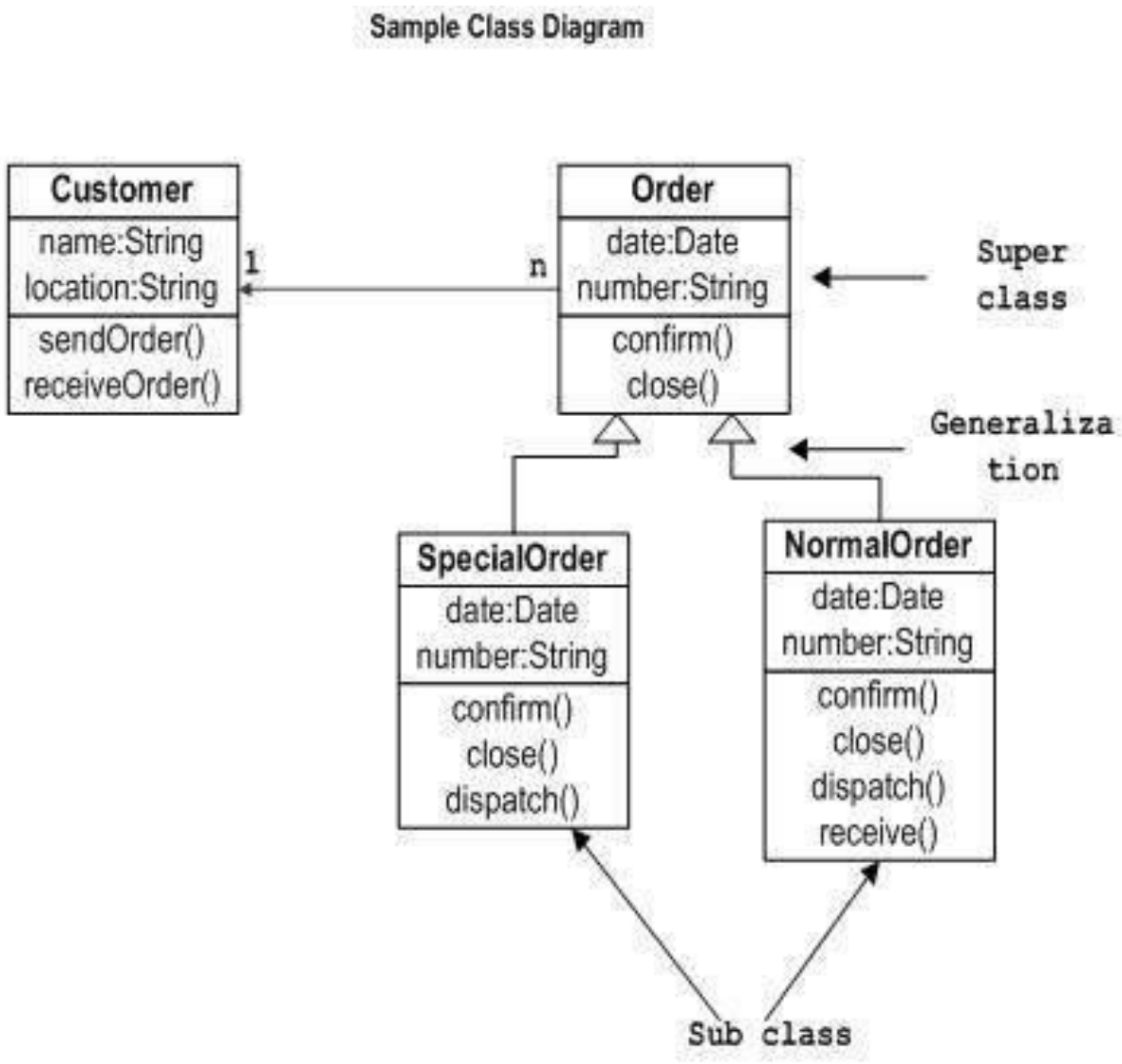
CLASS DIAGRAMS

- a static diagram
- most popular UML diagram in the coder community.
- Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.
- Class diagram describes the attributes and operations of a class and also the constraints imposed on the system.
- shows a collection of classes, interfaces, associations, collaborations, and constraints.
- Purpose
  - Analysis and design of the static view of an application.
  - Describe responsibilities of a system.
  - Base for component and deployment diagrams.
  - Forward and reverse engineering.

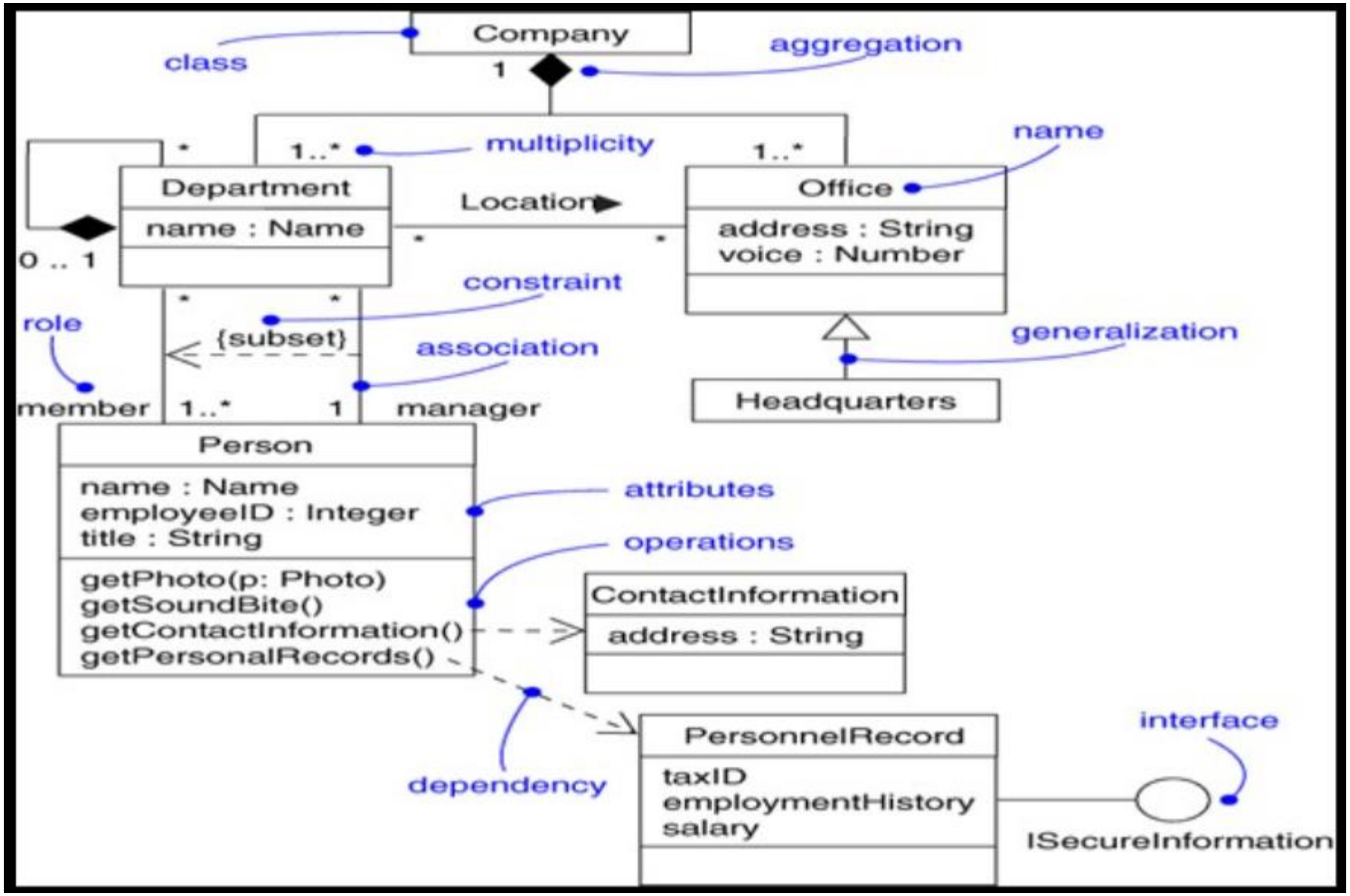
CLASS NOTATION



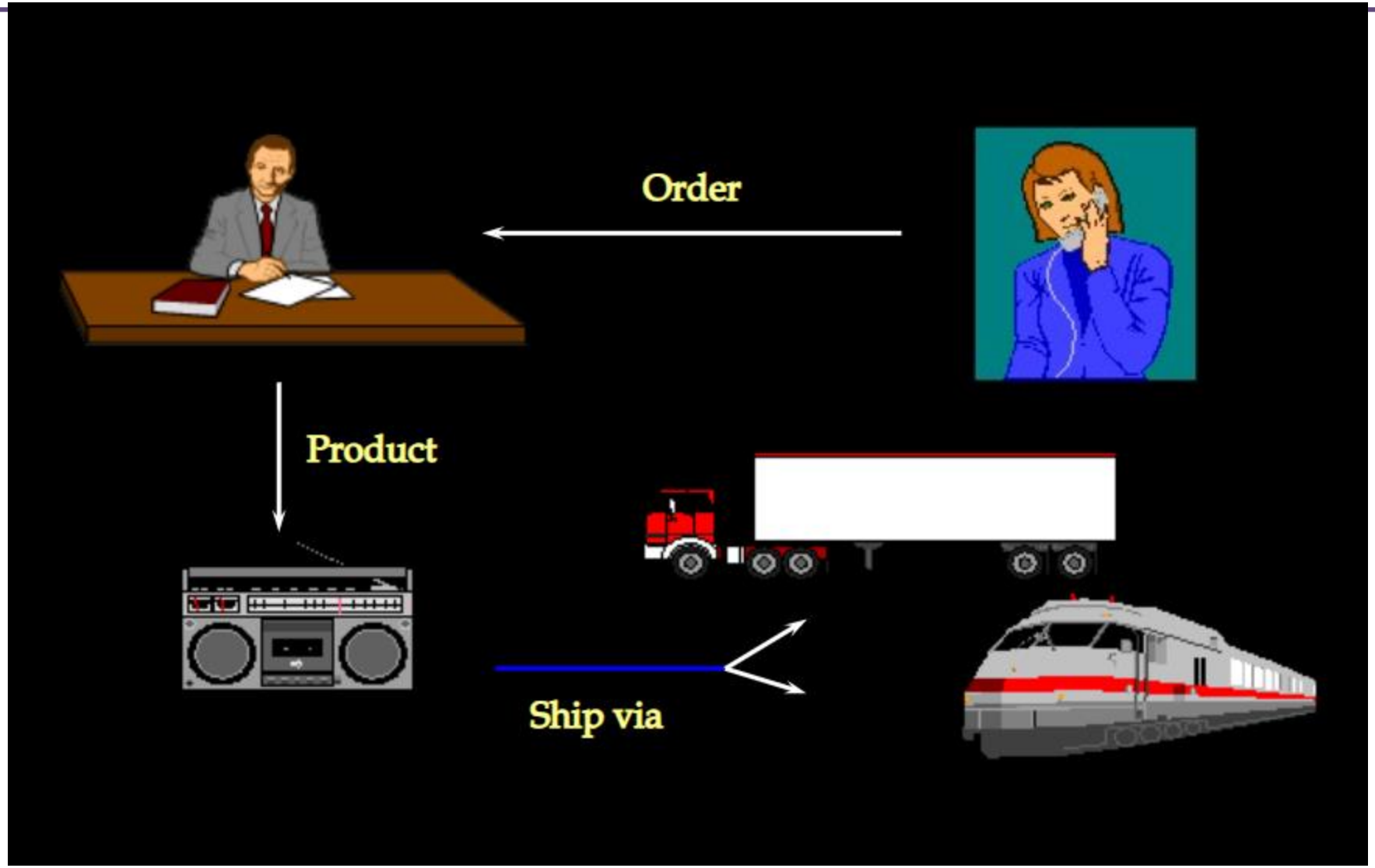
CLASS DIAGRAMS: EXAMPLE-1



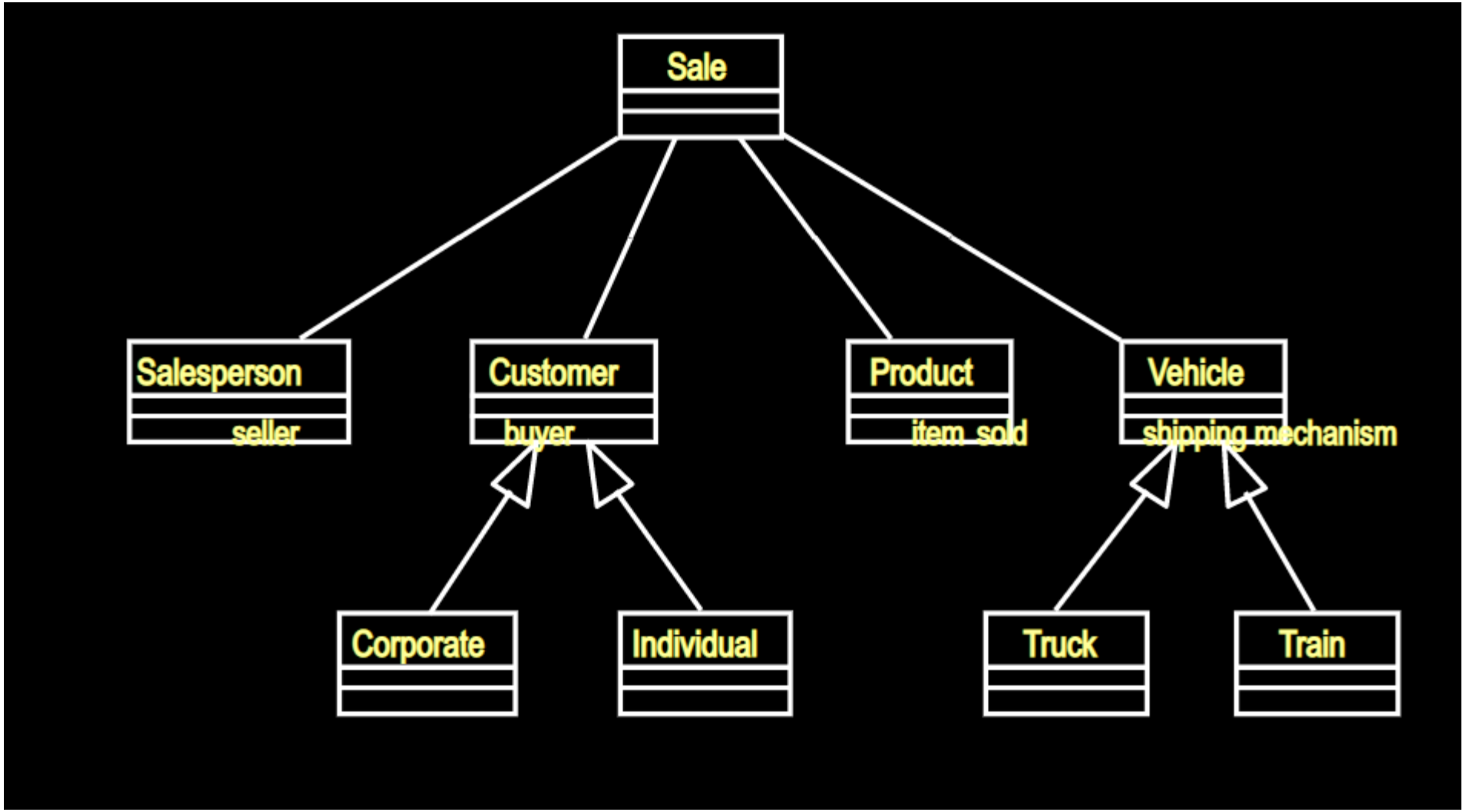
CLASS DIAGRAMS: EXAMPLE-2



CLASS DIAGRAMS: EXAMPLE-3



CLASS DIAGRAMS: EXAMPLE-3

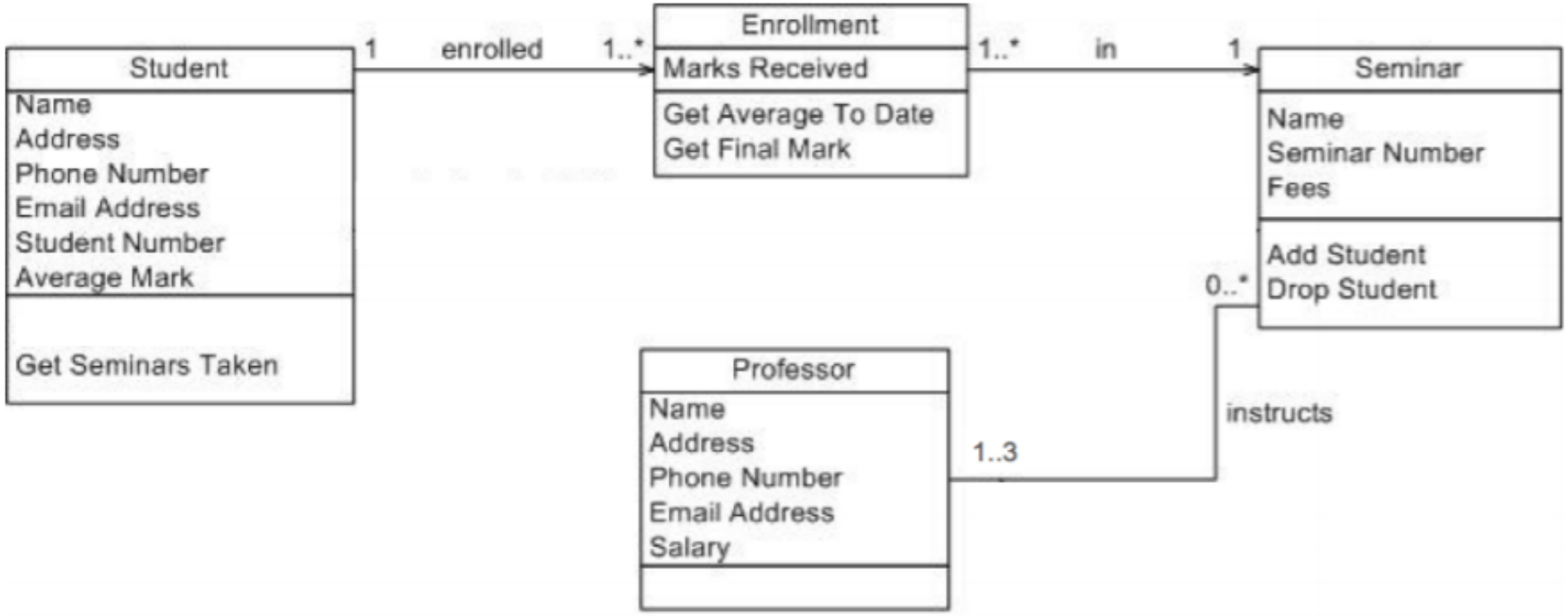


CLASS DIAGRAMS: EXERCISE 1

Example: A professors has a name, address, phone number, email address, and salary. A student has also a name, etc., but no salary (sorry). A student, however, has an average mark (of the final marks of his or her seminars). A seminar has a name and a number. When a student is enrolled in a seminar, the marks for this enrollment are recorded and the current average as well as the final mark (if there is one) can be obtained from the enrollment. From a student one can obtain a list of seminars he or she is enrolled in. Professors teach seminars. Each seminar has at least one and at most three teachers. There are two types of seminar: bachelor and master. From a bachelor seminar students can not withdraw. From a master seminar they can.

Draw a class diagram for this university. Add attributes and methods when necessary.

CLASS DIAGRAMS: SOLUTION SAMPLE



USE CASE DIAGRAMS

- Capture the dynamic behavior
- Dynamic behavior means the behavior of the system when it is running/operating
- Consists of actors, use cases and their relationships
- A single use case diagram captures a particular functionality of a system
- A number of use case diagrams are needed for more understanding requirements
- Used to gather the requirements of a system including internal and external influences
- **Purposes:**
  - Used to gather the requirements of a system.
  - Used to get an outside view of a system.
  - Identify the external and internal factors influencing the system.
  - Show the interaction among the requirements are actors.

USE CASE DIAGRAMS – EXAMPLE 1

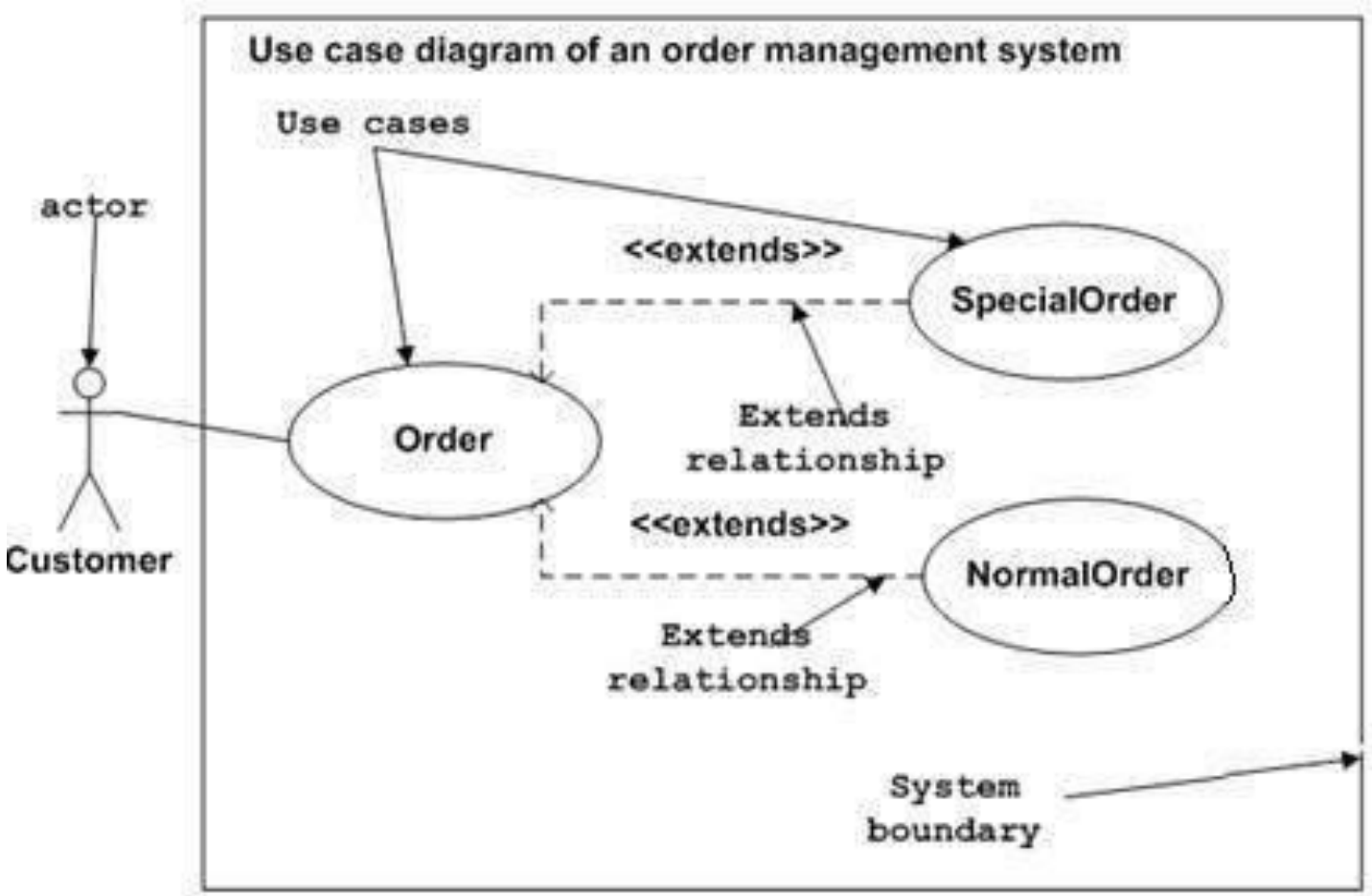
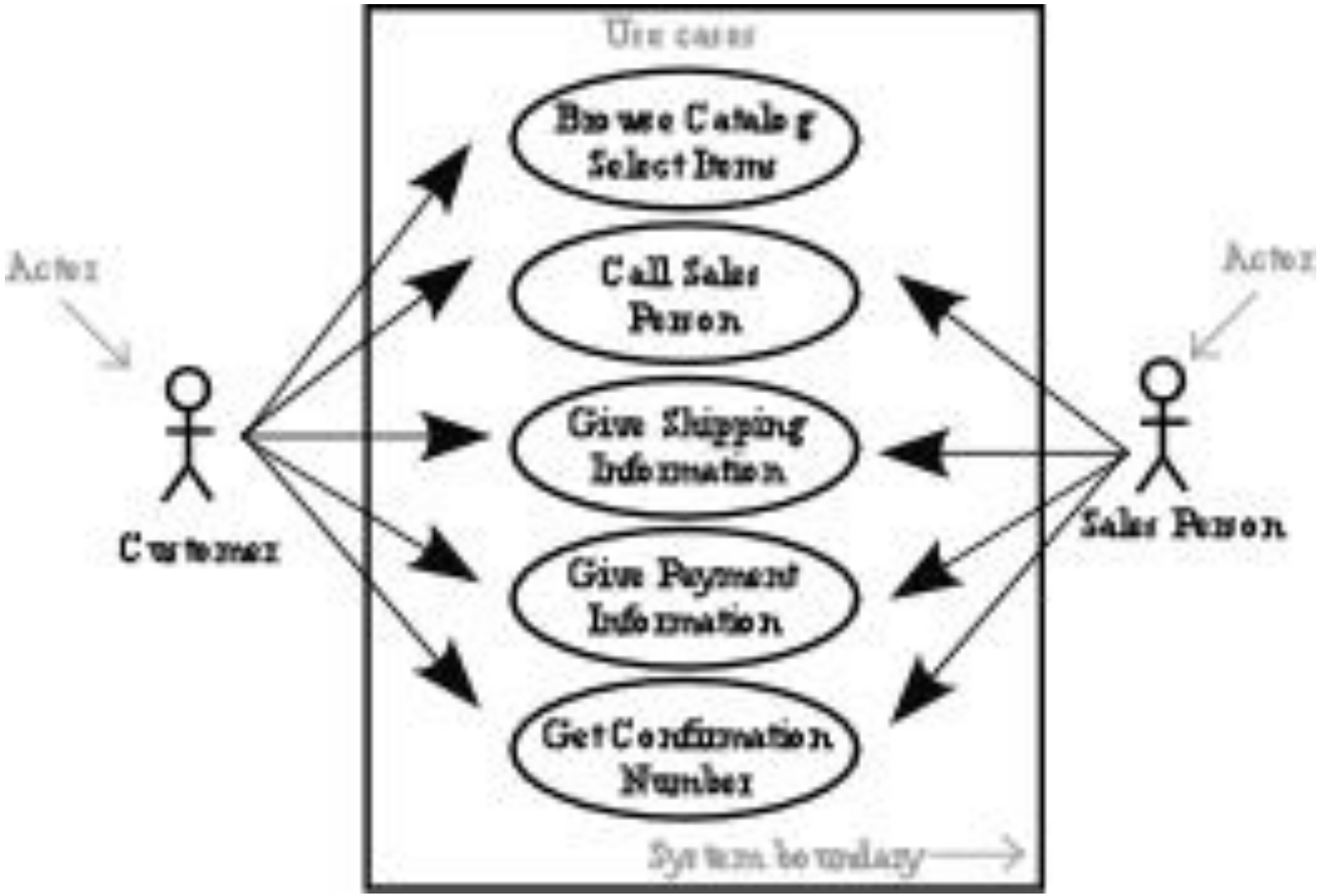


Figure: Sample Use Case diagram

USE CASE DIAGRAMS – EXAMPLE 2



USE CASE DIAGRAMS – EXERCISE

Consider the following required functionality of a print-on-demand service:

The print-on-demand service provides customers the possibility to print posters, flyers, or books on demand. The customer should be able to select a type of product (poster, flyer, or book), a desired quantity, and a paper type. In case a book has to be printed, additionally the customer can choose between hard cover and soft cover. Finally, the customer needs to provide a PDF file containing the desired content.

In order for the customer to be able to place an order, he or she must have an account. The customer can create an account by choosing a username/password combination. Furthermore, his or her address and credit card number can be linked to the account, which is required information when placing an order.

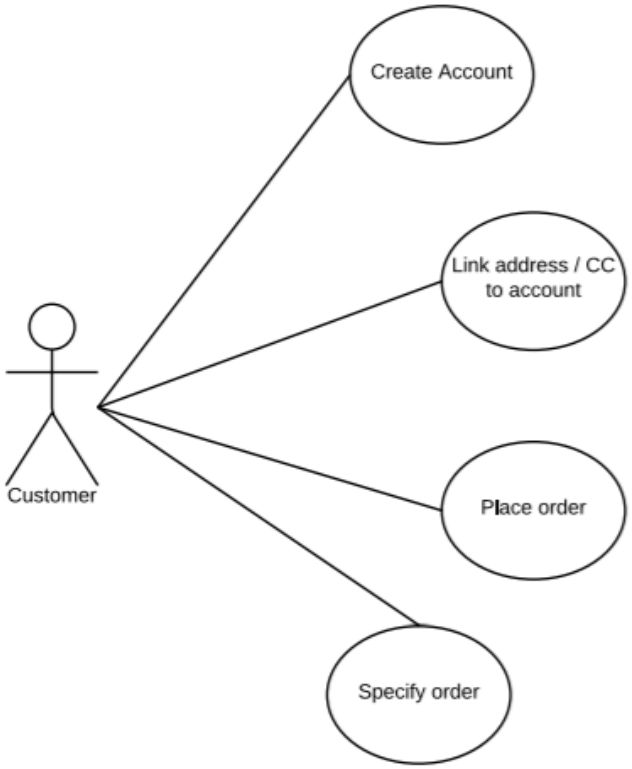
Once a customer has provided the information for an order, the system checks if all required information is there, either given in the order (type of product, quantity, etc.), or in the account (address and payment information). If any information is lacking, the system will inform the customer that it needs to be added before the order can be placed. Once all information is in place, the order is placed, and the credit card information is sent to the bank for approval. If the bank approves the card, the order is finalized.

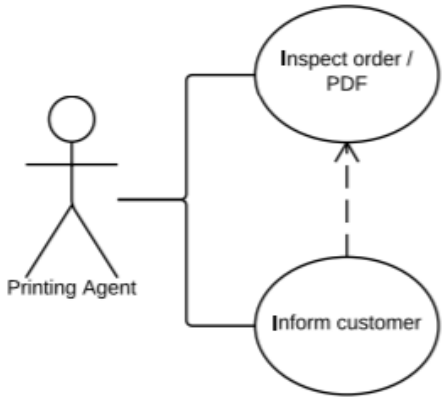
A printing agent is in charge of actually performing the printing. He or she inspects the provided PDF files of finalized orders. If a file does not meet the quality requirements, the customer will be informed about this, and the order is temporarily put on hold until the customer has provided a new PDF file.

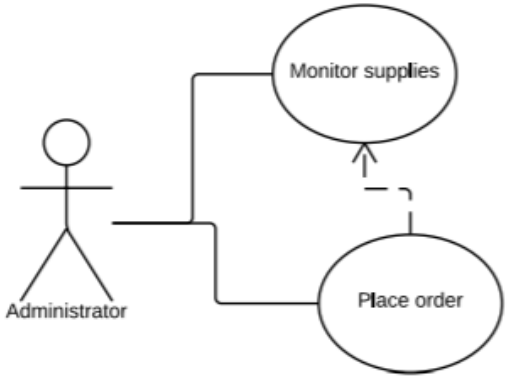
Finally, the administrator monitors if at all times, sufficient paper and ink stock is present. Whenever the amount of paper or ink is running low, an order must be placed at the appropriate supplier (either the paper or ink supplier).

Draw Use Case diagrams

USE CASE DIAGRAMS – SAMPLE SOLUTIONS







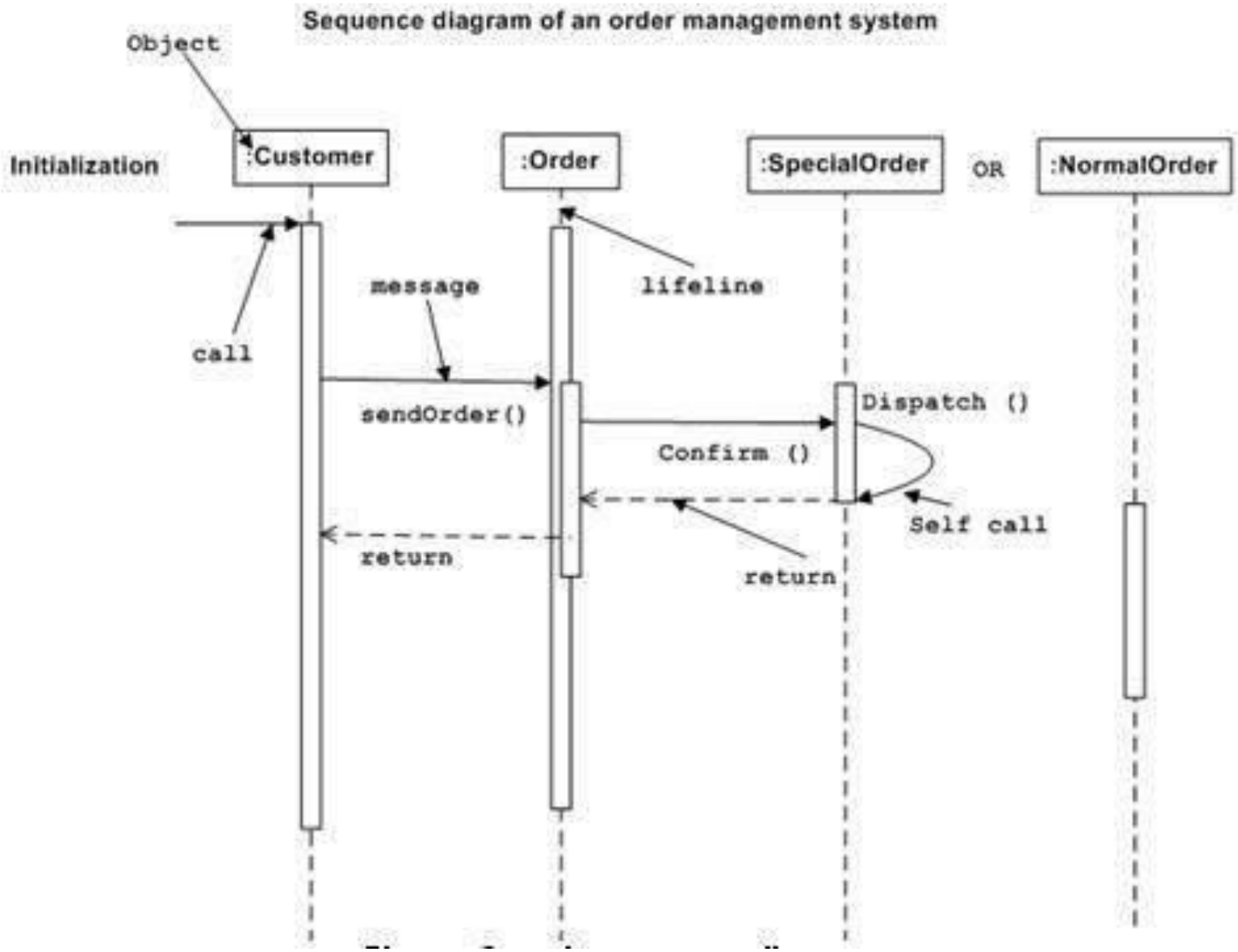
INTERACTION DIAGRAMS

- To describe some type of interactions among the different elements in the model
- Purposes:
  - To capture the dynamic behaviour of a system.
  - To describe the message flow in the system.
  - To describe the structural organization of the objects.
  - To describe the interaction among objects.
- Two interaction diagrams
  - Sequence diagram
  - Collaboration diagram

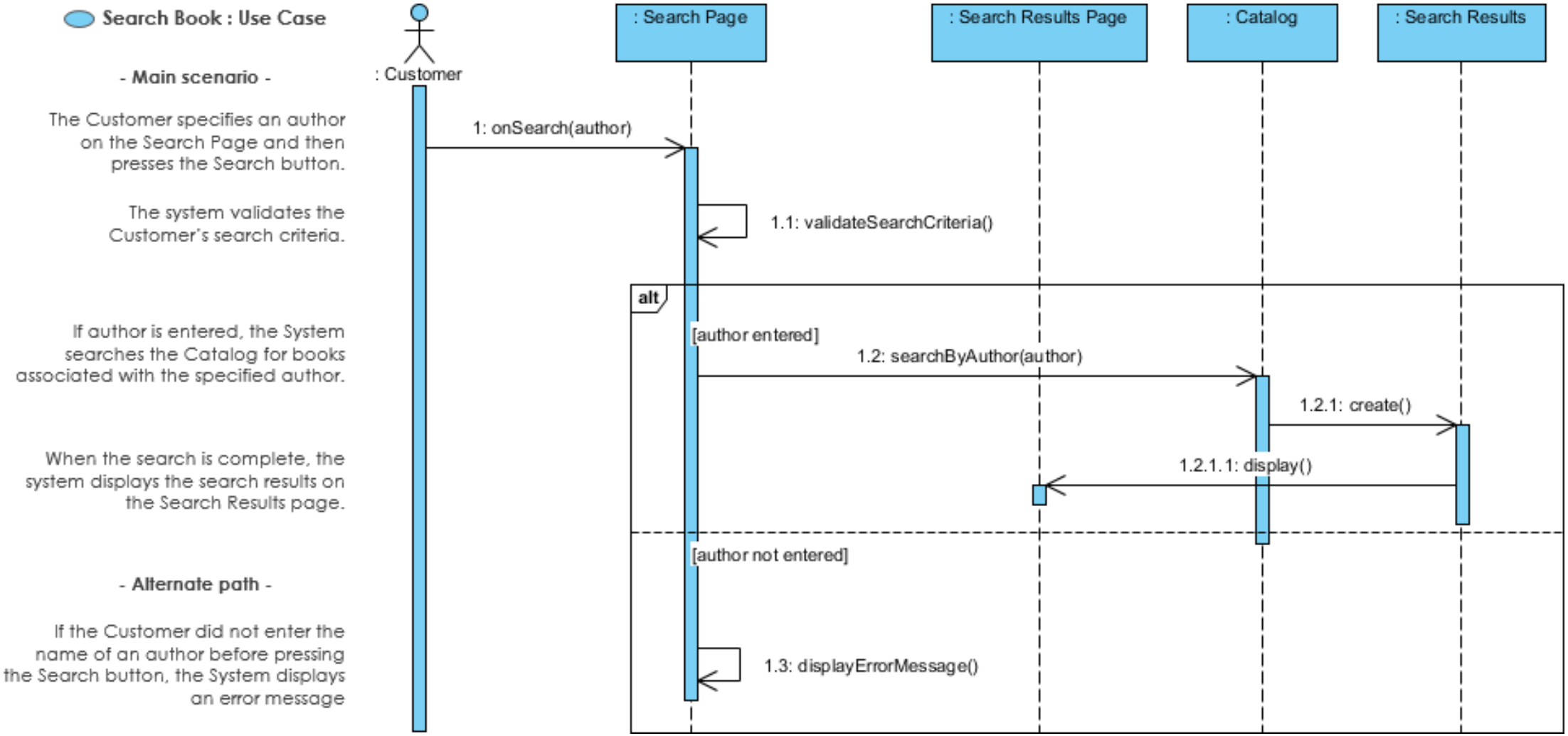
SEQUENCE DIAGRAMS

- the interaction that takes place in a collaboration that either realizes a use case or an operation (instance diagrams or generic diagrams)
- high-level interactions between user of the system and the system, between the system and other systems, or between subsystems (sometimes known as system sequence diagrams)
- **Purposes:**
  - Model high-level interaction between active objects in a system
  - Model the interaction between object instances within a collaboration that realizes a use case
  - Model the interaction between objects within a collaboration that realizes an operation
  - Either model generic interactions (showing all possible paths through the interaction) or specific instances of a interaction (showing just one path through the interaction)

SEQUENCE DIAGRAMS – EXAMPLE 1



SEQUENCE DIAGRAMS – EXAMPLE 2



https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/

SEQUENCE DIAGRAMS – EXCECISE

Consider the following required functionality of a print-on-demand service:

The print-on-demand service provides customers the possibility to print posters, flyers, or books on demand. The customer should be able to select a type of product (poster, flyer, or book), a desired quantity, and a paper type. In case a book has to be printed, additionally the customer can choose between hard cover and soft cover. Finally, the customer needs to provide a PDF file containing the desired content.

In order for the customer to be able to place an order, he or she must have an account. The customer can create an account by choosing a username/password combination. Furthermore, his or her address and credit card number can be linked to the account, which is required information when placing an order.

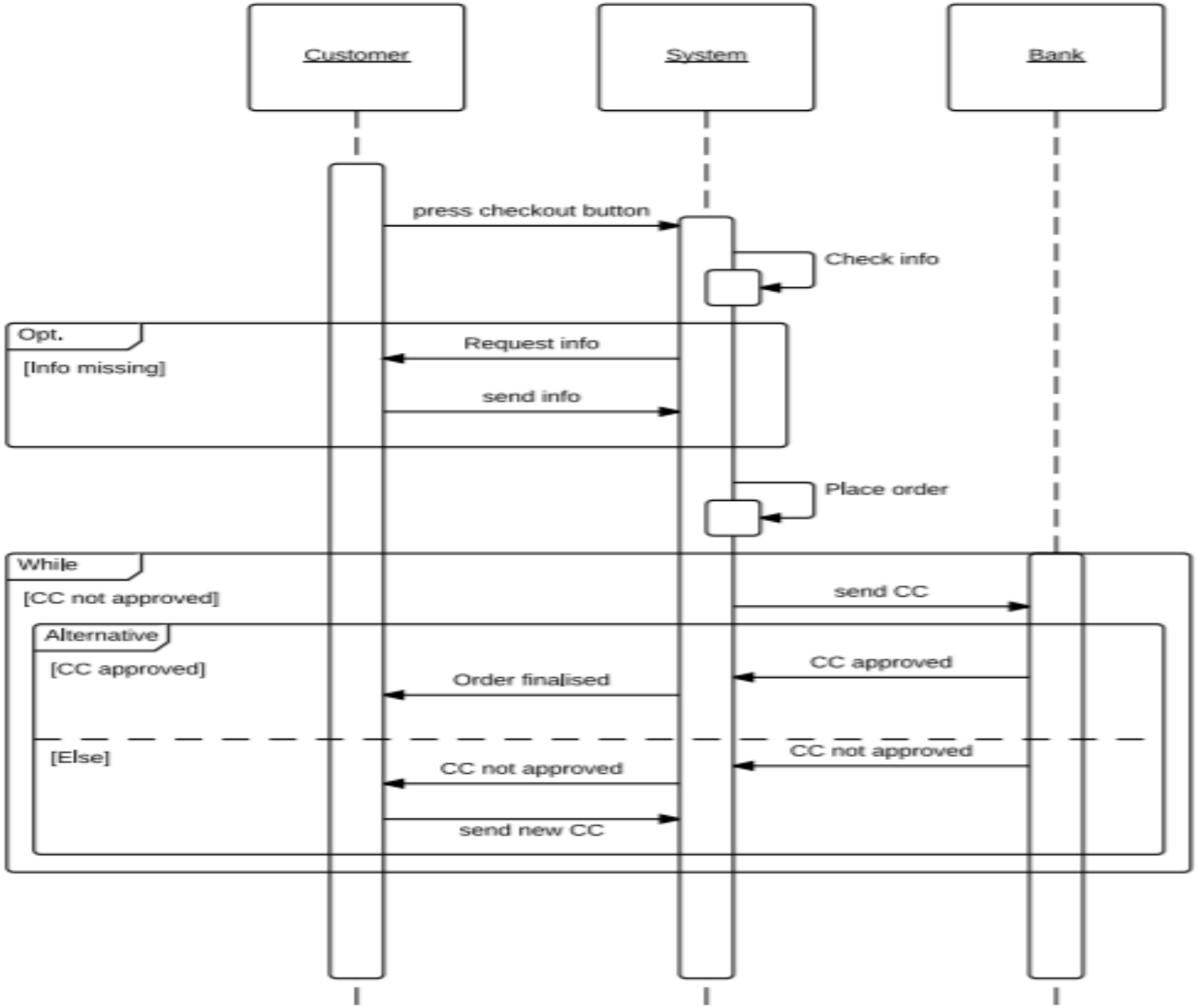
Once a customer has provided the information for an order, the system checks if all required information is there, either given in the order (type of product, quantity, etc.), or in the account (address and payment information). If any information is lacking, the system will inform the customer that it needs to be added before the order can be placed. Once all information is in place, the order is placed, and the credit card information is sent to the bank for approval. If the bank approves the card, the order is finalized.

A printing agent is in charge of actually performing the printing. He or she inspects the provided PDF files of finalized orders. If a file does not meet the quality requirements, the customer will be informed about this, and the order is temporarily put on hold until the customer has provided a new PDF file.

Finally, the administrator monitors if at all times, sufficient paper and ink stock is present. Whenever the amount of paper or ink is running low, an order must be placed at the appropriate supplier (either the paper or ink supplier).

Draw sequence diagram

SEQUENCE DIAGRAMS – SAMPLE SOLUTION

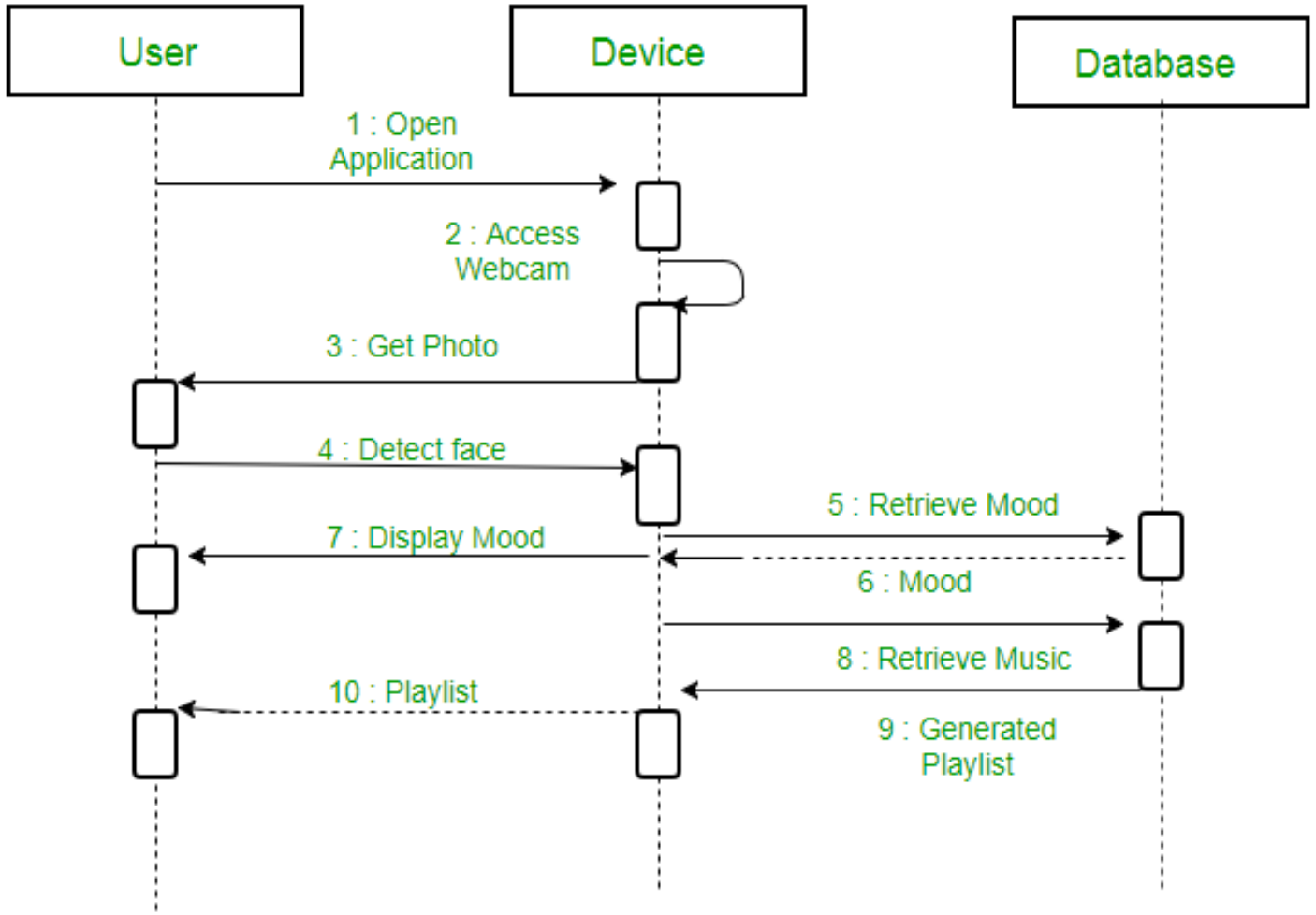


Exercise

Let us consider an emotion-based music player system which play the relevant music by detecting user's mood. When user open the application, application will use webcam and take picture of user. Then, application detect the face and predict the user's mood and mood is shown to user. Then by using the rule-based system, which is store in database, application will generate a list of songs based on user's mood and presented to system.

Based on above scenario, make a sequence diagram with proper notation.

Solution Sample

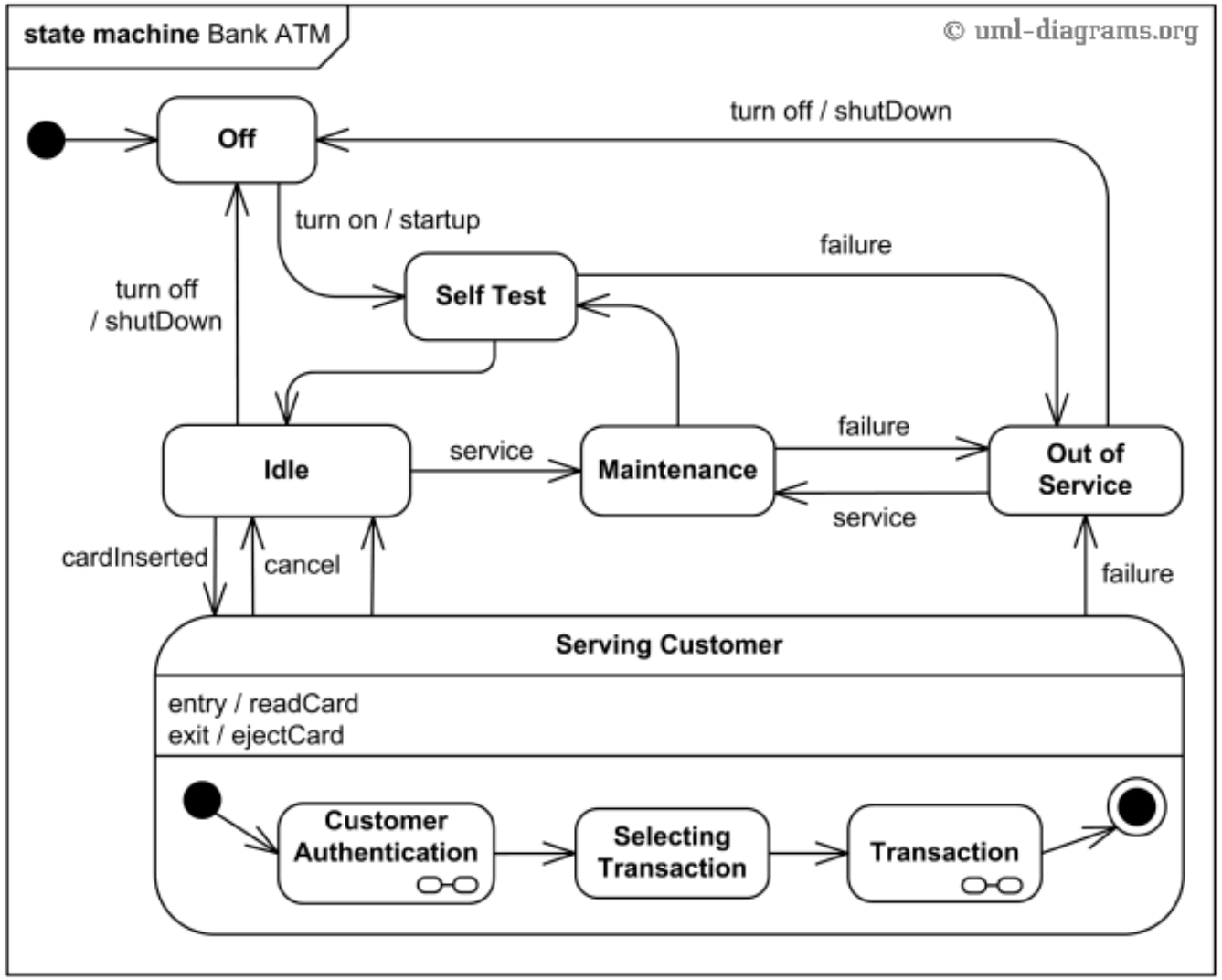


1. Firstly, the application is opened by the user.
2. The device then gets access to the web cam.
3. The webcam captures the image of the user.
4. The device uses algorithms to detect the face and predict the mood.
5. It then requests database for dictionary of possible moods.
6. The mood is retrieved from the database.
7. The mood is displayed to the user.
8. The music is requested from the database.
9. The playlist is generated and finally shown to the user.

STATE MACHINE DIAGRAMS

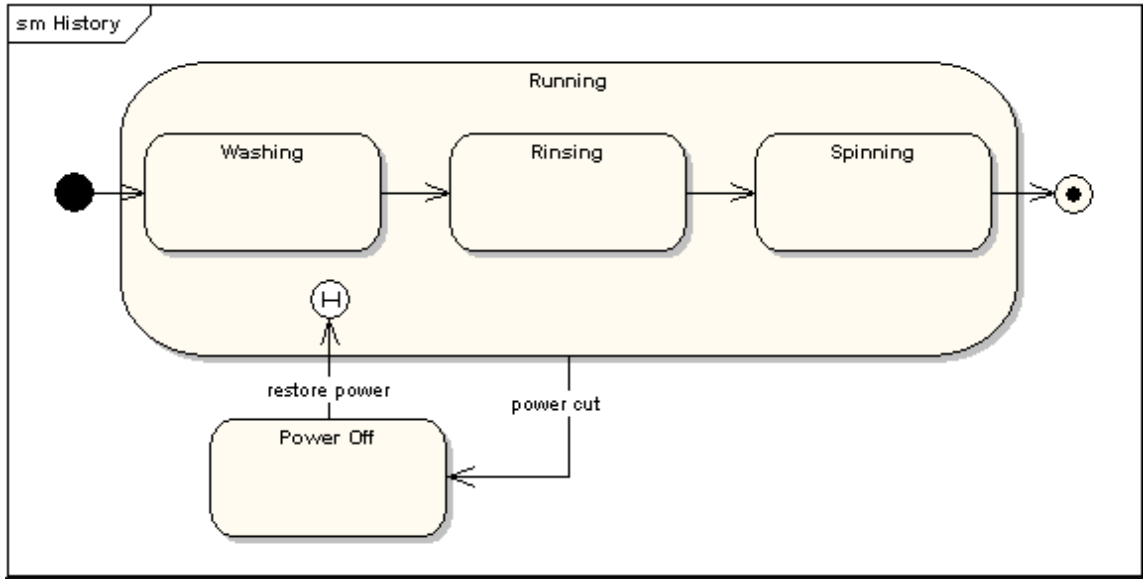
- Describes different states and transitions for a component or object in a system.
- The states represent various conditions or modes in which a component exists during its lifecycle.
- Also referred to as a state diagram, state machine diagram, or state chart diagram.
- A state machine defines a finite set of states, transitions, and events that determine state changes.
- Activity diagrams (explained separately) are a specialized form of state machine diagrams.
- **Key Characteristics of a State Machine:**
  - *States*: Conditions or situations during the life of an object.
  - *Events*: Triggers that cause transitions between states.
  - *Transitions*: Movement from one state to another due to an event.
  - *Initial and Final States*: Represent the starting point and termination of the state machine.
- **Purposes:**
  - To model the dynamic behavior of a system or component.
  - To illustrate the lifecycle of a reactive object responding to events.
  - To describe different states of an object and how it transitions between those states over time.
  - To define a state machine that governs the behavior and state changes of a system element.

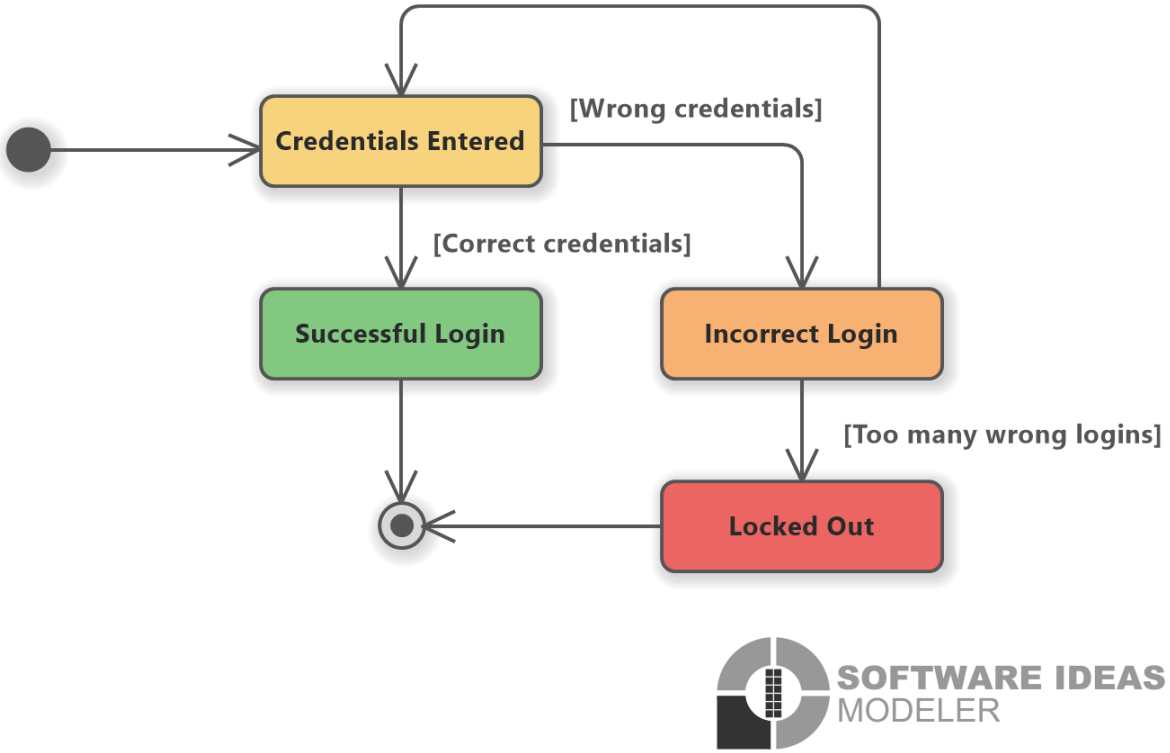
STATE MACHINE DIAGRAMS



Bank ATM

STATE MACHINE DIAGRAMS

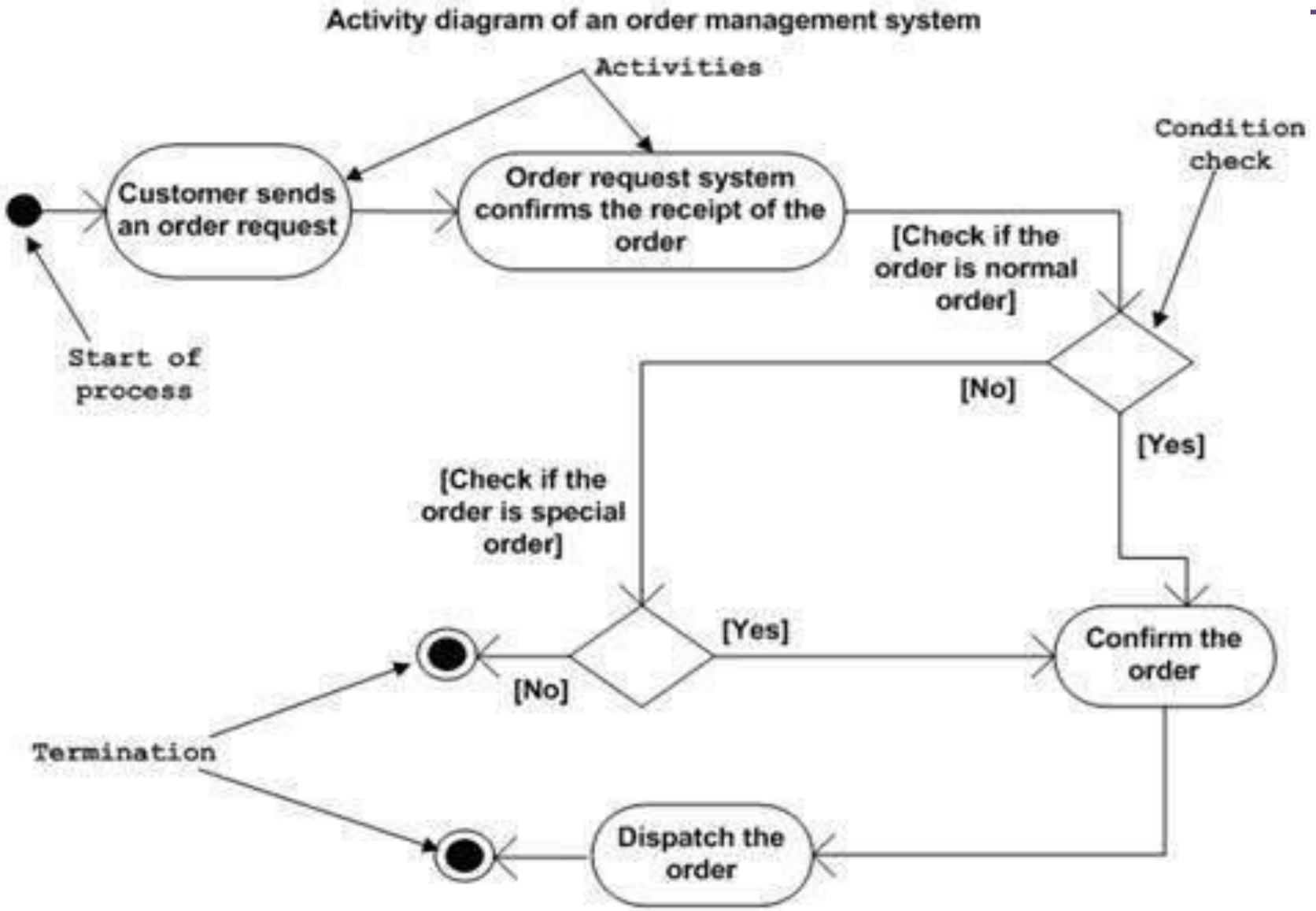




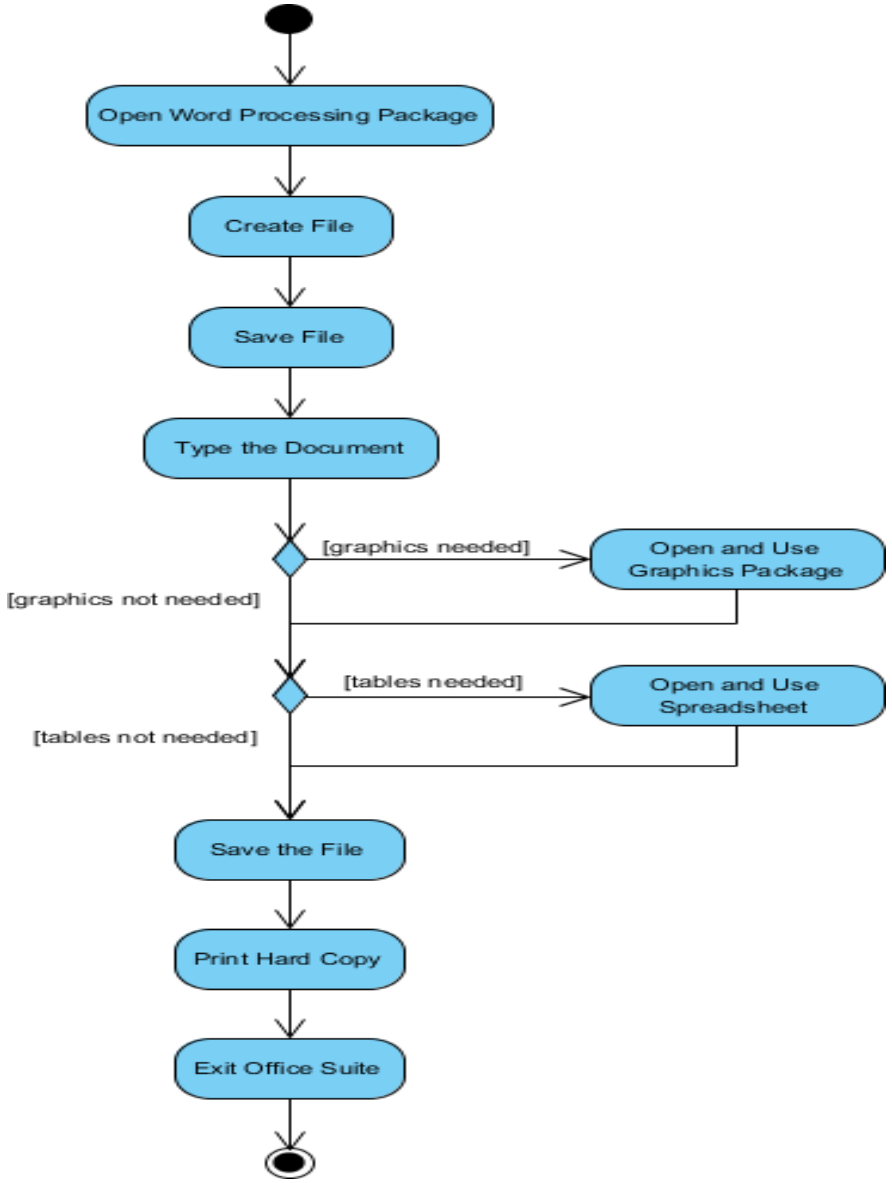
ACTIVITY DIAGRAMS

- A flowchart to represent the flow from one activity to another activity
- Flowchart like diagrams
- Activity is a particular operation of the system
- Flow can be sequential, branched, or concurrent
- **Purposes:**
  - Draw the activity flow of a system.
  - Describe the sequence from one activity to another.
  - Describe the parallel, branched and concurrent flow of the system.

ACTIVITY DIAGRAMS – EXAMPLE 1

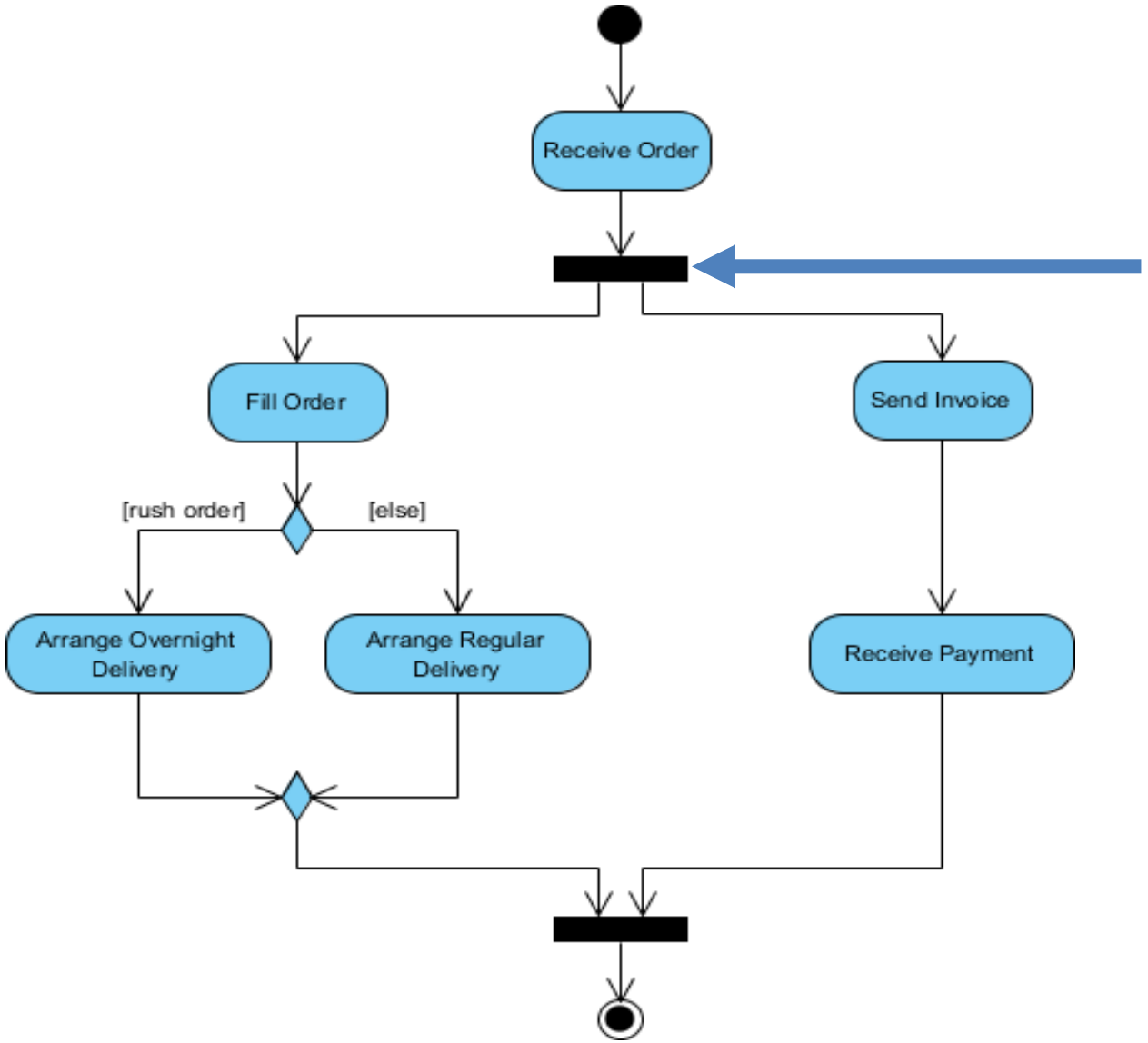


ACTIVITY DIAGRAMS – EXAMPLE 2



https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/

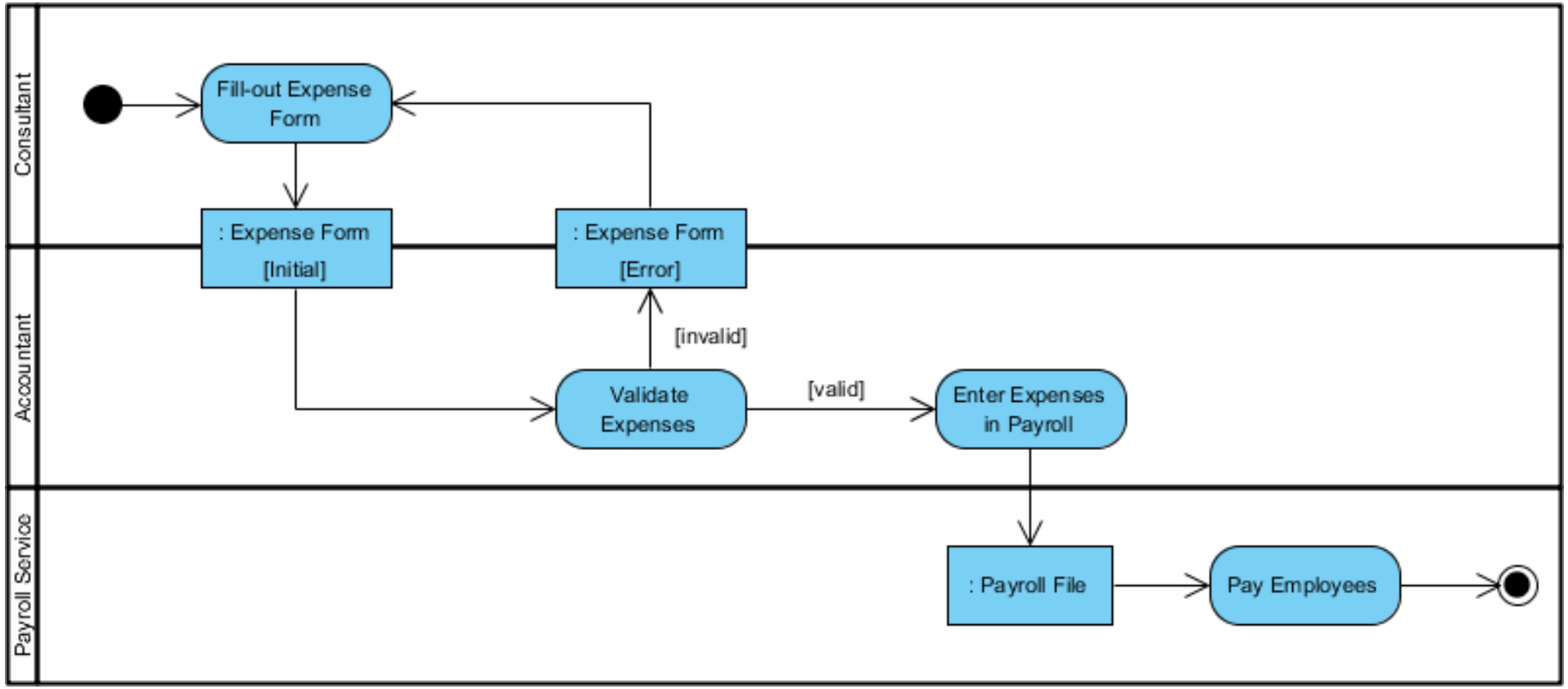
ACTIVITY DIAGRAMS – EXAMPLE 3



Parallel set of activities

https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/

ACTIVITY DIAGRAMS – EXAMPLE 4



Activity Diagram - Swinlane: A swimlane is a way to group activities performed by the same actor on an activity diagram or activity diagram or to group activities in a single thread. This is an example of a swinlane activity diagram for modeling Staff Expenses Submission.

https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/

ACTIVITY DIAGRAMS – EXERCISE

Consider the following required functionality of a print-on-demand service:

The print-on-demand service provides customers the possibility to print posters, flyers, or books on demand. The customer should be able to select a type of product (poster, flyer, or book), a desired quantity, and a paper type. In case a book has to be printed, additionally the customer can choose between hard cover and soft cover. Finally, the customer needs to provide a PDF file containing the desired content.

In order for the customer to be able to place an order, he or she must have an account. The customer can create an account by choosing a username/password combination. Furthermore, his or her address and credit card number can be linked to the account, which is required information when placing an order.

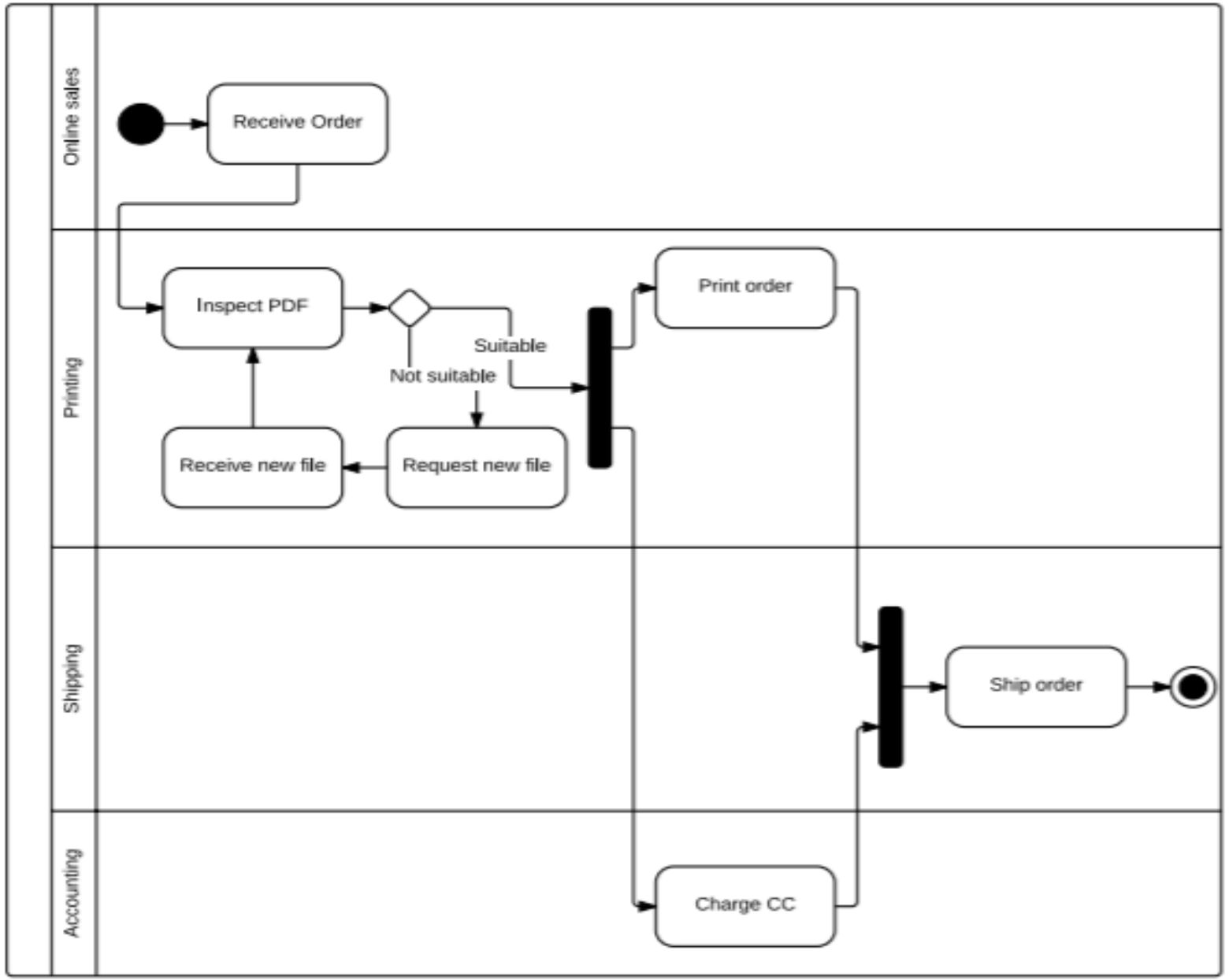
Once a customer has provided the information for an order, the system checks if all required information is there, either given in the order (type of product, quantity, etc.), or in the account (address and payment information). If any information is lacking, the system will inform the customer that it needs to be added before the order can be placed. Once all information is in place, the order is placed, and the credit card information is sent to the bank for approval. If the bank approves the card, the order is finalized.

A printing agent is in charge of actually performing the printing. He or she inspects the provided PDF files of finalized orders. If a file does not meet the quality requirements, the customer will be informed about this, and the order is temporarily put on hold until the customer has provided a new PDF file.

Finally, the administrator monitors if at all times, sufficient paper and ink stock is present. Whenever the amount of paper or ink is running low, an order must be placed at the appropriate supplier (either the paper or ink supplier).

Draw activity diagram

ACTIVITY DIAGRAMS – SAMPLE SOLUTION



Modeling Interactions and Object Lifecycles

Modeling Interaction and Object Lifecycle

- Modelling Interactions: Focuses on how objects collaborate to achieve a use case.
- Object Lifecycles: Depicts the sequence of states an object goes through from creation to destruction.
- *Purpose*:
  - Understand dynamic behavior in systems.
  - Represent object collaborations and state transitions.

Mapping Use Case to Object

- Use Cases: Describe functional requirements from the user's perspective.
- Steps to Map Use Cases:
  1. Identify key scenarios and actors.
  2. Determine participating objects and their roles.
  3. Define messages exchanged between objects.
- Example:
  - Use Case: "Place an Order"
  - Objects: Customer, Order, Product, Payment.

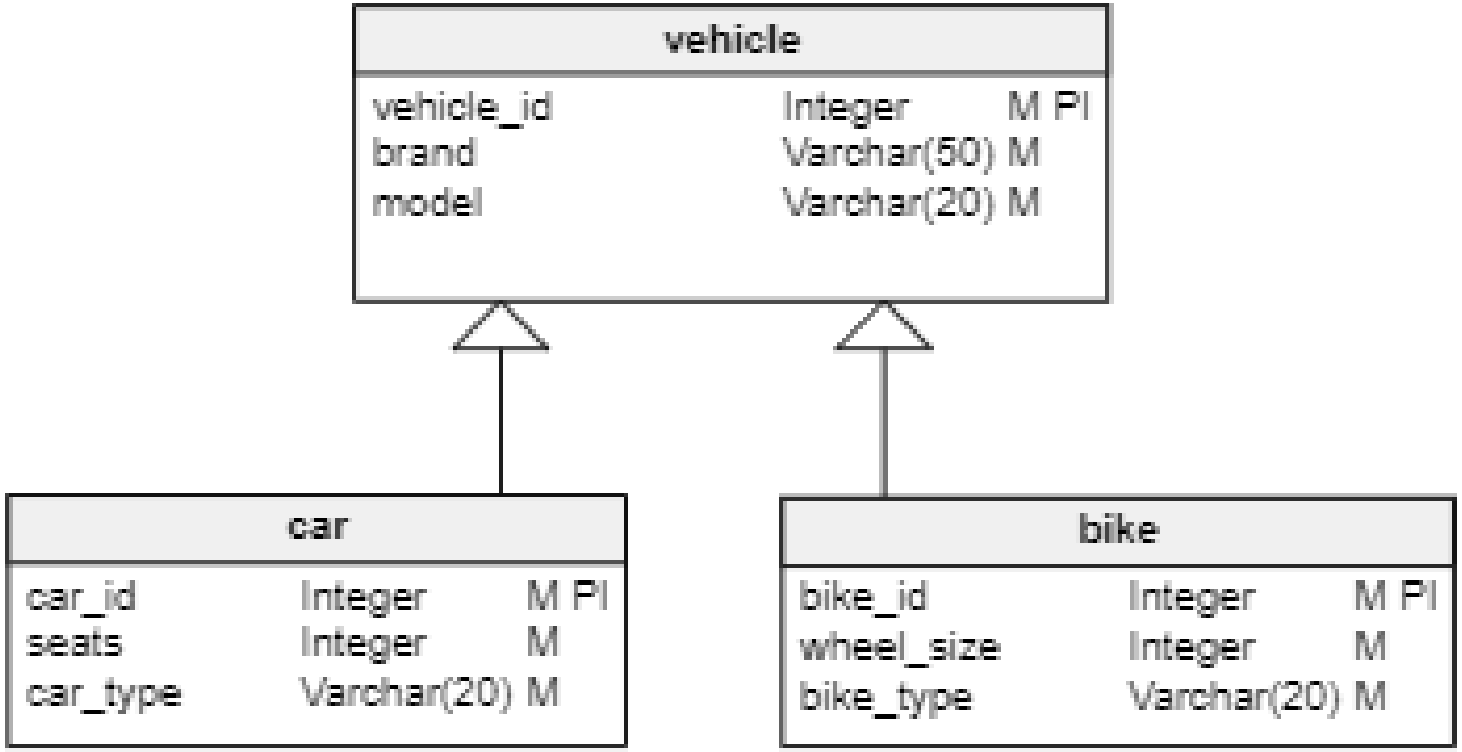
Modeling Associations and Aggregations

- Associations: Define relationships between two or more objects.
  - Example: A Student enrols in a Course.
- Aggregations: Represent a whole-part relationship where parts can exist independently.
  - Example: A Library contains multiple Books.

Modeling Inheritance Relationships

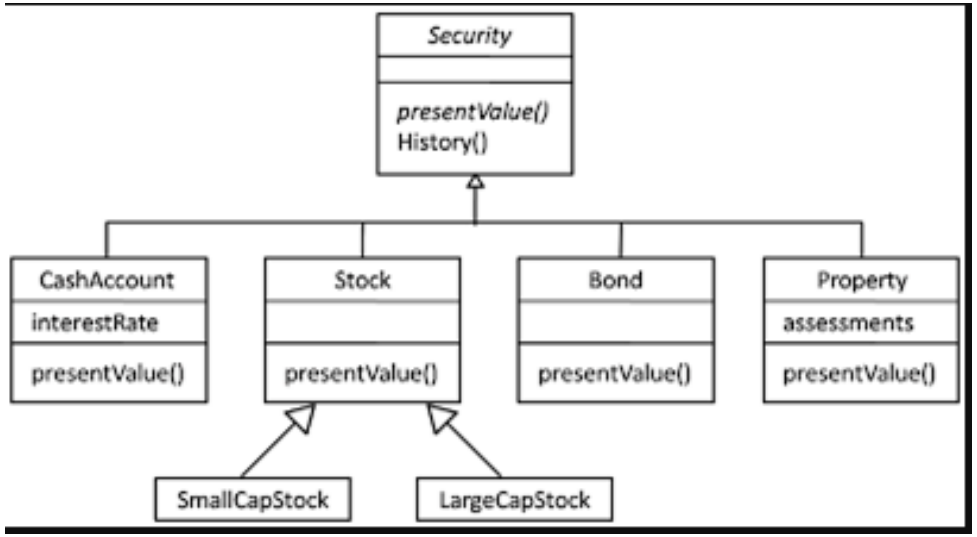
- Both iterative refinement and client sign-off procedures are vital in ensuring high-quality analysis and design models.
- It ensures that all requirements are met, design decisions are sound, and the project aligns with client expectations.
- While iterative refinement fosters flexibility and continuous improvement, client sign-off provides formal approval and accountability, ensuring that both the development team and the client are aligned before progressing.

Modeling Inheritance Relationship



Modeling Inheritance relationship in database

Modeling Inheritance Relationship



Modeling Inheritance relationship

Reviewing Analysis and Design Models

Reviewing Analysis and Design Models

- Both iterative refinement and client sign-off procedures are vital in ensuring high-quality analysis and design models.
- It ensures that all requirements are met, design decisions are sound, and the project aligns with client expectations.
- While iterative refinement fosters flexibility and continuous improvement, client sign-off provides formal approval and accountability, ensuring that both the development team and the client are aligned before progressing.

Iterative Refinement

- Process
  1. *Initial Draft*: Create an initial version of the model based on initial requirements.
  2. *Stakeholder Feedback*: Share the model with stakeholders and gather input.
  3. *Refinement*: Update the model based on feedback, fixing errors or incorporating new ideas.
  4. *Revalidation*: Present the revised model for another round of feedback.
  5. *Finalization*: Continue iterations (2 to 4) until all parties agree the model meets the requirements.
- Benefits
  - Reduces the risk of major issues later in development.
  - Allows flexibility to adapt to changes in requirements.
  - Promotes early identification of design flaws.

Client Sign-Off Procedure

- Client sign-off procedures ensure that analysis and design models are formally approved by the client before moving to the next phase of development.
- **Key Features**
  - **Formal Agreement**: A documented acknowledgment from the client that the models meet their expectations and requirements.
  - **Clear Milestones**: Sign-offs are typically required at specific stages, such as after requirement analysis or design completion.
  - **Accountability**: Ensures all parties are aligned and reduces the likelihood of disputes later.

Client Sign-Off Procedure

- **Process**:
  - **Presentation of Models**: The finalized version of the models is presented to the client.
  - **Review by Client**: The client reviews the models to confirm they meet the agreed-upon requirements.
  - **Documentation**: A formal document is prepared for the client to review, sign, and approve the models.
  - **Feedback (if needed)**: If the client requests changes, the models are revised and resubmitted.
  - **Sign-Off**: The client provides formal approval, signalling the readiness to proceed to the next phase.
- **Benefits**:
  - Establishes a clear boundary between phases of the project.
  - Minimizes scope creep by freezing requirements post-sign-off.
  - Ensures the client has a complete understanding of what will be developed.

END OF UNIT 04